

UNIVERSIDAD TÉCNICA FEDERICO SANTA MARÍA
DEPARTAMENTO DE INGENIERÍA QUÍMICA Y AMBIENTAL
SANTIAGO-CHILE



**Aplicación de Redes Neuronales Artificiales para mejorar la
descripción de la dispersión espacial y temporal de la humedad en la
lixiviación de cobre**

Francisco José Zúñiga Ginestar

MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL QUÍMICO

PROFESOR GUÍA: Daniel Navia

PROFESOR CORREFERENTE: Iván Cornejo

Marzo - 2025



CONSTANCIA DE VALIDACIÓN Y CONFIDENCIALIDAD DE MONOGRAFÍA A REPOSITORIO ACADÉMICO

1.- IDENTIFICACIÓN DEL TRABAJO ACADÉMICO

Tipo de monografía (marcar una opción): ☒ Memoria o trabajo de título ☐ Tesis de Postgrado

Título del trabajo: Aplicación de Redes Neuronales Artificiales para mejorar la descripción de la dispersión espacial y temporal de la humedad en la lixiviación de cobre

Nombre del candidato(a): Francisco José Zúñiga Ginestar

Carrera / Grado: Ingeniero Civil Químico

Campus: Casa San Joaquín Departamento: Ingeniería Civil Química Ambiental

2.- VALIDACIÓN DEL PROFESOR GUÍA/DIRECTOR DE TESIS

Yo, DANIEL ANDRÉS NAVIA LÓPEZ, en mi calidad de profesor(a) guía/director(a) del trabajo académico mencionado anteriormente **DEJO CONSTANCIA** que:

- He revisado esta versión del documento y corresponde a la versión final aprobada del trabajo.
- El trabajo cumple con los requisitos académicos y de formato establecidos por la institución.

3.- EVALUACIÓN DE CONFIDENCIALIDAD POR PROPIEDAD INDUSTRIAL (marcar una opción)

☒ El trabajo **NO contiene** información que amerite confidencialidad y puede ser publicado de inmediato en repositorio con acceso abierto.

☐ El trabajo **CONTIENE** información con potenciales implicancias de propiedad industrial o intelectual y requiere un periodo de confidencialidad (**embargo**) por (**marcar una opción**):

☐ 6 meses ☐ 12 meses ☐ 2 años ☐ 3 años ☐ 5 años ☐ 10 años

Fundamentación de la necesidad de confidencialidad (obligatorio si se solicita embargo):

4.- FIRMAS

Profesor(a) guía o director(a) de memoria o tesis:

Fecha: 14 DE OCTUBRE DE 2025

Firma: 

Estudiante o Candidato(a):

Fecha: 14/10/2025

Firma: 

Este formulario debe ser insertado como página 2 de la memoria o tesis, completado y firmado por estudiante y profesor(a) antes de la entrega en portal PRISMA de Biblioteca USM.

RESUMEN EJECUTIVO

Este trabajo de título se enfocó en describir la dispersión espacial y temporal de la humedad en una columna de lixiviación mediante el uso de modelos fenomenológicos y redes neuronales artificiales (ANN). El objetivo general fue integrar redes neuronales dentro de un modelo fenomenológico basado en la ecuación de Richards, para determinar las variables de conductividad y difusividad hidráulicas y así obtener una predicción de la humedad en la columna de lixiviación.

Se utilizaron datos simulados de humedad, conductividad hidráulica K y difusividad hidráulica D generados a partir de curvas de retención de Van Genuchten. Estos datos se emplearon para entrenar redes neuronales en dos configuraciones: una red con dos salidas (para K y D) y dos redes separadas, cada una con una salida para K y D . Las redes se entrenaron utilizando el algoritmo L-BFGS, y su desempeño se evaluó mediante métricas como curvas de pérdida, el coeficiente de determinación R^2 , gráficos de dispersión e histogramas de error.

Los resultados mostraron que al utilizar una red para K y otra red para D se obtuvo un desempeño superior dentro de la simulación, con un R^2 cercano a 1, lo que indica una alta precisión en las predicciones. Por el contrario, la red con dos salidas presentó un R^2 menor a 0.5, lo que se puede explicar por un sesgo sistemático en las predicciones debido a su arquitectura.

En conclusión, este trabajo demostró que las redes neuronales en conjunto con el modelo de Richards y especialmente aquellas con una salida para cada variable, son una herramienta capaz de realizar la descripción de la humedad en pilas de lixiviación, siendo una alternativa viable para la simulación matemática.

Esto tiene implicaciones significativas para la optimización mediante la programación matemática del proceso, ya que, al reemplazar los modelos de curva de retención tradicionales por modelos diferenciables como lo son las ANN, no es requerido el uso de ecuaciones que aportan discontinuidad al modelo debido a su estructura matemática, no son diferenciables y conllevan un error estructural de las ecuaciones. Sin embargo, al ser las redes entrenadas con datos de estas curvas de retención, estas tienen a su vez una incertidumbre estructural implícita proveniente del sesgo de la obtención de los datos.

Se recomienda explorar el uso de redes neuronales informadas por procesos físicos (PINN) en futuras investigaciones para mejorar aún más la precisión y robustez del modelo implementado, además de ocupar datos reales de K y D para el entrenamiento de redes neuronales profundas DNN, para mejorar la certeza de los datos predichos.

ÍNDICE

1. INTRODUCCIÓN	1
1.1. Proceso de lixiviación en pilas	1
1.2. Escenario actual de la lixiviación en Chile	3
1.3. Estado del arte	4
2. OBJETIVOS	6
2.1. Objetivo general	6
2.2. Objetivos específicos	6
3. TEORÍA	7
3.1. Bases teóricas del modelo	7
3.1.1. Caracterización de suelos	7
3.1.2. Ley de Darcy	8
3.1.3. Ecuación de Richards	9
3.1.4. Conductividad hidráulica	10
3.2. Modelo Fenomenológico	12
3.3. Machine Learning	12
3.4. Redes neuronales Artificiales	13
3.4.1. Neuronas artificiales	13
3.4.2. Capas	15
3.4.3. Función de pérdida	15
3.4.4. Arquitectura Compleja	17
3.4.5. Redes neuronales basadas en procesos físicos (PINN)	17
4. METODOLOGÍA	18
4.1. Determinación de hiperpárametros de las Redes neuronales	18
4.2. Estrategia de entrenamiento e incorporación al modelo	21
5. RESULTADOS Y DISCUSIÓN	26
5.1. Determinación de Hiperpárametros de las redes neuronales	26
5.2. Resultados del entrenamiento e integración de redes neuronales	27
5.2.1. Validación redes neuronales	28
5.2.2. Comparación entre modelos	35
5.3. Discusión sobre resultados	37
5.3.1. Implementación del método matemático y problemática	38

5.3.2.	Definición del algoritmo de entrenamiento.....	41
6.	Conclusiones y recomendaciones	43
7.	Referencias	45
8.	Anexos	48
8.1.	Anexo A, parámetros físicos utilizados y datos utilizados	48
8.2.	Anexo B, Resultados sensibilización de hiperparametros	49
8.3.	Anexo C, Crank-Nicolson Modificado	49
8.4.	Anexo D, Método de Euler explícito	51
8.5.	Anexo E, Método de Runge-Kutta	52
8.6.	Anexo F, Reformulación de la integración de las redes al modelo fenomenológico.....	54
8.7.	Anexo G, Adimensionalización del modelo de Richards	55
8.8.	Anexo H, Código sensibilización de hiperparametros	56
8.9.	Anexo I, Código entrenamiento y optimización DNN.....	59
8.10.	Anexo J, Código comparación de modelos DNN	61
8.11.	Anexo K, Código entrenamiento y optimización PINN	64
8.12.	Anexo L, Código comparación de modelos matemáticos	67
8.13.	Anexo M, Código Implementación PINN alternativo	71

ÍNDICE DE FIGURAS

Figura 1:	Diagrama de flujo del proceso de lixiviación en pilas.....	2
Figura 2:	Curvas de retención en condiciones de secado, humectación y zona media para un suelo parcialmente saturado (Fredlund, 2012).....	8
Figura 3:	Permeámetro de carga constante (Sánchez, 2024).	8
Figura 4:	Contenido de humedad para modelos de van Genuchten y exponencial (Tracy, 2006). ..	11
Figura 5:	Diagrama Red neuronal Artificial. (Gerón, 2019).....	14
Figura 6:	Dominio matemático del modelo (Cariaga et al, 2015).....	19
Figura 7:	Diagrama de flujo de incorporación del modelo (elaboración propia).....	21
Figura 8:	Algoritmo de entrenamiento DNN	23
Figura 9:	Comparación entre modelo con VGM y modelo con DNN (elaboración propia).....	24
Figura 10:	Gráficos de dispersión para datos de entrenamiento	26
Figura 11 :	Gráficos de predicción de K y D respectivamente.	26
Figura 12:	Desempeño de entrenamiento y validación <i>RedJ</i>	28
Figura 13:	Gráficos de dispersión de la <i>RedJ</i> .para la variable K, el grafico de la izquierda corresponde a los datos de validación y el de la derecha corresponde a los datos de entrenamiento.	28
Figura 14:	Histogramas de error de <i>RedJ</i> para la variable K, el histograma de la izquierda corresponde a los datos de validación y el histograma de la derecha corresponde a los datos de entrenamiento.	29

Figura 15: Gráficos de dispersión de la <i>RedJ</i> .para la variable D, el grafico de la izquierda corresponde a los datos de validación y el de la derecha corresponde a los datos de entrenamiento.	29
Figura 16: Histogramas de error de <i>RedJ</i> para la variable D, el histograma de la izquierda corresponde a los datos de validación y el histograma de la derecha corresponde a los datos de entrenamiento.	30
Figura 17: Desempeño de entrenamiento y validación <i>Redd</i> y <i>Redk</i> respectivamente.	31
Figura 18: Gráficos de dispersión de la <i>Redk</i> .para la variable K, el grafico de la izquierda corresponde a los datos de validación y el de la derecha corresponde a los datos de entrenamiento.	31
Figura 19: Gráficos de dispersión de la <i>Redk</i> .para la variable K, el grafico de la izquierda corresponde a los datos de validación y el de la derecha corresponde a los datos de entrenamiento.	32
Figura 20: Gráficos de dispersión de la <i>Redd</i> .para la variable D, el grafico de la izquierda corresponde a los datos de entrenamiento y el de la derecha corresponde a los datos de validación.	32
Figura 21: Gráficos de dispersión de la <i>Redd</i> .para la variable D, el grafico de la izquierda corresponde a los datos de validación y el de la derecha corresponde a los datos de entrenamiento.	33
Figura 22: Histograma de distribución de valores de Se en el intervalo	34
Figura 23: Gráficos de dispersión de la <i>RedJ</i> , <i>Redk</i> y <i>Redd</i> .para la variable Se, el grafico de la izquierda corresponde a la comparación entre el modelo base y el modelo usando la red <i>RedJ</i> , mientras que el de la derecha corresponde a la comparación entre el modelo base y el modelo usando las redes <i>Redk</i> y <i>Redd</i> .	36
Figura 24: Algoritmo de comparación modelos matemáticos	40
Figura 25: algoritmo de entrenamiento PINN (elaboración propia)	41

1. INTRODUCCIÓN

En la minería del cobre hay dos grandes procesos productivos de este mineral, la pirometalurgia y la hidrometalurgia. Este trabajo de título, se enfocará en la segunda vía de producción de este mineral. La vía de la hidrometalurgia consta de tres procesos principales necesarios para obtener cátodos de cobre: Lixiviación (LX), Extracción por solvente (SX) y Electro-obtención (EW).

El proceso en de la lixiviación, consiste en una extracción solido-líquido. En este proceso el solvente líquido es usualmente ácido sulfúrico diluido con iones de hierro, el cual permea a través del mineral de interés, extrayendo el cobre del mineral aglomerado sólido. El cobre retenido en la solución va a parar a un estanque previo al siguiente proceso. Existen diversos métodos de lixiviación como lo son por agitación, bateas inundadas, en pilas, en botaderos e in-situ.

El proceso de extracción por solvente es un proceso de purificación y concentración de la solución rica en cobre. El mineral de cobre el cual constituye como la sustancia soluble en ambas fases, se distribuye preferencialmente hacia una segunda fase líquida con la que entra en contacto, siendo las fases de origen y destino completamente inmiscibles entre ellas.

Por último, el proceso de Electro-obtención, consta de descomposición de la solución rica en cobre para extraer el metal mediante reducción electrolítica (Domic Mihovilovic, 2001). El resultado de este proceso es una lámina de cobre con un 99.99% de grado de pureza.

El presente trabajo se centrará en la lixiviación y la descripción espacial y temporal de la humedad en una columna para aumentar la recuperación de cobre y disminuir la posibilidad de inestabilidad mecánica desde una perspectiva fluidodinámica.

1.1. Proceso de lixiviación en pilas

El proceso de lixiviación consiste en la extracción del compuesto de interés desde un medio sólido hacia un medio líquido, mediante el uso de un agente lixivante.

En la pila su construcción comienza usando un sistema de apilador móvil, el cual consiste en una estructura metálica articulada la cual está montada sobre un sistema de desplazamiento de orugas y guiado por GPS, esta estructura descarga el aglomerado para construir la pila, estas pilas pueden tener desde 6 a 10 metros de altura (Domic Mihovilovic, 2001).

Una vez construida la pila de lixiviación, esta se rocía con un agente lixivante el cual para el caso de la minería del cobre se usa una solución acuosa con ácido sulfúrico para extraer el cobre. Es en esta etapa donde el agente percola dentro de la estructura de mineral aglomerado.

Dentro de la pila, ocurren dos fenómenos principales, la transferencia de masa y la fluidodinámica. A continuación, se dará una breve explicación de estos fenómenos.

Debido al fenómeno de la transferencia de masa ocurren interacciones entre los tres principales componentes de la lixiviación: el mineral aglomerado, la solución lixivante que es usualmente una solución diluida de ácido sulfúrico, y el aire atmosférico dentro del lecho de lixiviación. En este fenómeno la principal interacción que se busca favorecer es la disolución del mineral de cobre

presente en el medio acuoso ácido, esto con el fin de extraer el cobre del mineral proveniente de la mina.

El segundo fenómeno dentro de la pila de lixiviación es la fluidodinámica asociada a la percolación de la solución lixivante a través del lecho. Este fenómeno involucra el movimiento del líquido dada la permeabilidad del lecho, donde se busca que la permeabilidad sea alta, y que el paso del líquido por el sólido sea uniforme, lo que aumenta la extracción de cobre en el proceso. Cabe mencionar que la cantidad del agente lixivante que percola dentro de la estructura dependerá directamente de la razón de regado, que es el flujo del fluido por área afectada.

Por otro lado, los suelos porosos pueden tener tres estados posibles, seco, saturado o parcialmente saturado. En el estado seco, existe un sistema sólido-aire, donde solo se encuentra mineral y aire atmosférico dentro de la pila. En el estado saturado, existe un sistema sólido-líquido, donde se asume que todo el volumen de aire entre el mineral aglomerado fue remplazado por el líquido lixivante. Y, por último, en el estado parcialmente saturado tenemos un sistema sólido-líquido-aire, donde están presentes en las 3 fases de la lixiviación mencionados anteriormente.

Entonces, cuando se inicia el riego aumenta la cantidad de líquido disponible en la interfase con el mineral dentro del lecho, donde se busca llegar a un estado capilar o parcialmente saturado, donde las fuerzas encargadas de mantener la cohesión de la pila son presiones negativas en los poros, aun cuando no se alcance una saturación total del suelo.

Durante el regado de la pila, la solución irá permeando a través del lecho debido al efecto de la gravedad, generándose los estados anteriormente mencionados. Una vez que el agente lixivante haya pasado a través de la pila, el producto obtenido del proceso por el fondo es una solución rica en cobre llamada *Pregnant Leach Solution* (PLS), la cual es extraída por una membrana impermeable y un sistema de drenaje ubicado debajo de la pila previamente instalado en el terreno de construcción.

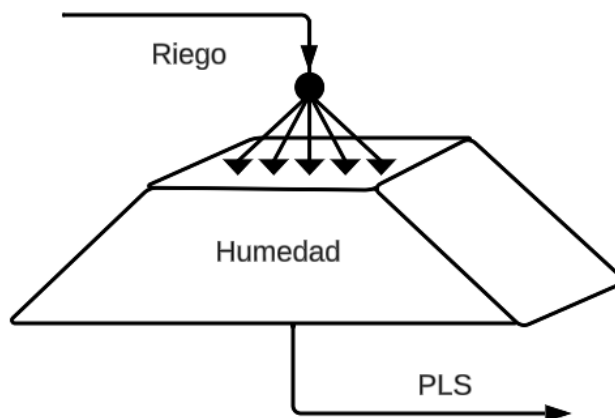


Figura 1: Diagrama de flujo del proceso de lixiviación en pilas

Es aquí donde surge la problemática de la saturación de agente lixivante en la pila, ya que, desde la perspectiva de la transferencia de masa, es deseable que aumente el área de contacto entre solución y mineral. Para lograr esto es necesario aumentar la saturación de la pila con solución lixivante y de esta forma aumentar la recuperación del mineral dentro del proceso.

Sin embargo, al excederse del límite de líquido o humedad de saturación crítica, se produce una suspensión sólido-líquido, lo que provoca que las fuerzas de tensión superficial que son las encargadas de mantener la cohesión de la pila dejen de estar presentes, lo cual conlleva a una disminución en la permeabilidad de la pila y pérdida de estabilidad mecánica de la pila.

Es por esto por lo que la investigación en parte se orienta al desarrollo de un sistema de control que pueda emplear el grado de saturación como variable manipulable y la búsqueda de ese grado de saturación óptimo dentro de la pila, para evitar estos efectos adversos y mejorar la recuperación general de cobre (Codelco EDUCA, 2019) (Adams B. A., 1994).

Se puede observar que esto constituye un cuello de botella para el proceso hidrometalúrgico, debido a que la totalidad del cobre extraído en esta etapa será el máximo de cobre posible a recuperar en fases posteriores, por lo tanto, es de suma importancia maximizar la recuperación de cobre en este proceso.

1.2. Escenario actual de la lixiviación en Chile

Actualmente, la industria minera del cobre en Chile es una de las más productivas del mundo, abarcando el 25% de la producción mundial de cobre y con una producción de 5.250,4 miles de toneladas métricas del mineral para el año 2023 (COCHILCO, 2024), del cual 1.334,5 miles de toneladas métricas corresponden a cátodos de cobre producidos mediante Extracción por solvente y Electro-obtención (SX-EW). Estos valores se encuentran por debajo de las proyecciones hechas para la producción nacional del cobre hecha para el 2028 por un margen del 9% aproximadamente.

Aun así, se proyecta para el 2028 una disminución en la producción de cátodos de cobre SX-EW del 23,2% (COCHILCO, 2023) y para el 2030 se espera una producción de tan solo 747.4 mil toneladas de cátodos de cobre producidos, siendo la mayoría del cobre producido en forma de concentrado para su exportación.

Por otro lado, también se experimenta un agotamiento en la ley del mineral en sulfuros y óxidos de cobre, y un aumento gradual de sulfuros secundarios y primarios de cobre, que son minerales refractarios a la lixiviación, lo que hace más desafiante la extracción de cobre.

Entonces la disponibilidad creciente de las instalaciones hidrometalúrgicas subutilizadas abre las oportunidades enfocadas al uso de estos minerales previamente no lixiviables, mediante innovaciones recientes en las técnicas de lixiviación como lo son el uso de cloruros y otros aditivos para lograr disolución de aquellas especies (Minería chilena, 2024).

Además, la hidrometalurgia es una de los procesos más versátiles para el tratamiento de otros tipos de minerales y obtención adicional de otros subproductos como lo son el oro, platino y la plata. Es por esto que resulta interesante el estudio de métodos de mejora y optimización de la recuperación del cobre desde esta perspectiva.

1.3.Estado del arte

El incremento en los costos de producción, el aumento de los estándares medioambientales, el agotamiento de las leyes de mineral en las minas y la mayor complejidad de los minerales extraídos han hecho necesario un rediseño del proceso de lixiviación, desde el desarrollo de pretratamientos, hasta el control operacional del proceso, junto con planificaciones y optimizaciones más precisas y robustas.

Esto con el objetivo de lograr recuperaciones de mineral consistentemente altas en un tiempo razonable y a bajo costo. Como lo señala David G. Dixon: “Las pilas de lixiviación son finalmente reconocidas por lo que siempre han sido: unidades de operación versátiles y complejas, que solo pueden ser diseñadas y operadas a su máximo potencial con la ayuda de herramientas de modelado sofisticadas” (Dixon, 2003).

Por otro lado, el avance de las nuevas tecnologías ha vuelto evidente la necesidad de implementar herramientas de inteligencia artificial para apoyar tareas como la predicción de la recuperación de mineral. Esto es crucial para que Chile mantenga su liderazgo en la industria minera y asegure ganancias a corto plazo (Víctor Flores, 2020).

Los modelos para describir las pilas de lixiviación pueden clasificarse en dos grandes categorías: modelos fenomenológicos y modelos basados en datos. Los modelos fenomenológicos presentan una alta complejidad matemática debido a la gran cantidad de factores que influyen en la distribución de la humedad o en la cantidad de solución lixivante dentro de la pila, como la altura de la pila, la conductividad hidráulica del suelo, la difusividad del mineral en el agente lixivante y la tasa de riego de la solución. Ejemplos de estos modelos incluyen:

- El modelo de Richards para describir la humedad.
- Modelos empíricos como los de Van Genuchten y Gardner para determinar la conductividad hidráulica y curva de retención.

Estos modelos establecen una relación entre el perfil de humedad y las variables fluidodinámicas que afectan al perfil de humedad. Estos modelos son fundamentales para el control preciso de la humedad lo que permitiría optimizar la extracción. No obstante, se presenta un desafío significativo: existe una discrepancia temporal entre los tiempos característicos de los fenómenos involucrados.

Mientras que la relación humedad-recuperación opera en escalas temporales de días, la relación humedad-tasa de riego, que es la variable de control, responde en rangos de minutos a horas. Esta diferencia en las dinámicas temporales complica sustancialmente la implementación de estrategias de control predictivo efectivas.

Esta asincronía temporal representa un obstáculo crítico para el desarrollo de sistemas de control automatizados que busquen optimizar la recuperación mineral mediante el ajuste dinámico de las tasas de riego en función de las mediciones de humedad.

Ante estas limitaciones, surge la posibilidad de adoptar un enfoque distinto para modelar la relación entre la humedad y el coeficiente de conductividad hidráulica mediante técnicas de machine learning.

Ejemplos de trabajos que utilizan este enfoque basado en datos incluyen los de (Sun et al., 2019), (Flores et al., 2020) y (Depina et al, (2022)), donde el principal enfoque de estos trabajos fue la utilización de modelos basados en datos para relacionar la recuperación de cobre con las variables del proceso de lixiviación utilizando diferentes metodologías como son los modelos de regresión de redes neuronales. Estos modelos ofrecen una alternativa prometedora para superar las limitaciones de los modelos tradicionales y mejorar la precisión y eficiencia en la predicción y control de las pilas de lixiviación.

2. OBJETIVOS

2.1.Objetivo general

- Describir el perfil espaciotemporal de la humedad dentro de las pilas de lixiviación mediante la integración de redes neuronales artificiales dentro de un modelo fenomenológico.

2.2.Objetivos específicos

- Definir tipo de Red Neuronal Artificial a utilizar dentro del modelo.
- Definir método de integración de las redes neuronales sobre la descripción fenomenológica de la humedad dentro de las pilas de lixiviación.
- Evaluar desempeño de la red neuronal respecto a capacidad de ajuste a datos entregados según los parámetros de número de neuronas y número de capas.

3. TEORÍA

3.1. Bases teóricas del modelo

Con el objetivo de utilizar la técnica de redes neuronales artificiales dentro de un modelo fenomenológico, y de esta manera describir la dispersión espacial y temporal de humedad dentro de una columna de lixiviación, es evidente que se deberá escoger un modelo para aplicar esta técnica.

Para este fin es necesario conocer las bases teóricas del caso de estudio para así comprender el problema y determinar donde se aplicará la técnica de redes neuronales artificiales. Como referencia se toman las bases teóricas expuestas por Cariaga et al. (2015)

3.1.1. Caracterización de suelos

Con el paso del tiempo, la composición química y las propiedades físicas del mineral extraído para la lixiviación cambian, lo que influye directamente en su interacción con la solución lixivante utilizada para su riego. Obtener estas variables permitirá modelar y predecir mejor la respuesta del lecho durante el proceso de riego. Los suelos típicamente se caracterizan de acuerdo con:

Tamaño de partícula: Previo al aglomerado el mineral pasa por un proceso de conminución o chancado, en el cual se busca obtener un tamaño de partícula entre 3/8 y 1/2 de pulgada (Dixon, 2003). Dado que es un proceso aleatorio, se obtiene una curva de distribución de tamaño de partícula o PSD por sus siglas en inglés, en la cual se identifica una frecuencia acumulada de las partículas que pasan por cierto tamiz de distinta apertura. El tamaño de partícula está directamente relacionado con la porosidad del lecho.

Porosidad: Responde al porcentaje de espacios vacíos del lecho en un volumen determinado. Es claro que, a mayor porosidad, el lecho podrá admitir mayor contenido de líquido.

Conductividad hidráulica: Es la capacidad del lecho de permitir o restringir el flujo de líquido a través del lecho. Tiene directa relación con la humedad o grado de saturación y la porosidad. A medida que aumenta la humedad, mayor es la conductividad del suelo. Existen ecuaciones para determinar su comportamiento, las cuales se detallarán más adelante.

Humedad: Para este trabajo, se utilizará humedad másica en base seca, que determina la relación entre la masa del líquido sobre la masa de sólido seco en un volumen de control dentro del lecho.

Grado de saturación: Otra forma de representar el contenido de líquido en una muestra, determinado como el porcentaje volumétrico de llenado de poros, respecto a una condición de saturación.

Curva de retención: Representa la relación entre el potencial de succión del suelo y su contenido volumétrico de líquido. Esto tiene directa relación con la película contráctil de líquido que se forma alrededor de las partículas sólidas en el lecho. En la práctica, esta curva tiene una forma sigmoidea y puede ser modelada para distintas condiciones del suelo como se muestra en la figura 2.

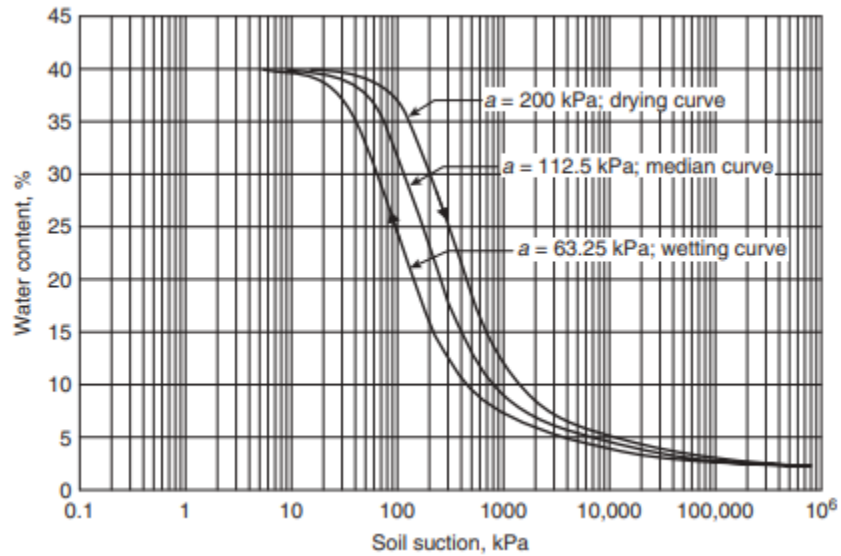


Figura 2: Curvas de retención en condiciones de secado, humectación y zona media para un suelo parcialmente saturado (Fredlund, 2012)

Existen diversos modelos matemáticos que se basan en el comportamiento de las curvas de retención para predecir la conductividad hidráulica, como los propuestos por van Genuchten (1980) y Gardner (1958). Estos modelos serán de gran utilidad al momento de plantear la simulación de la curva de retención y determinar el parámetro de conductividad hidráulica. Sin embargo, antes de profundizar en estos modelos, es importante comprender los orígenes del desarrollo del modelado de los fenómenos de percolación de fluidos.

3.1.2. Ley de Darcy

El flujo de agua a través de un lecho saturado puede ser descrito usando la ley de Darcy (1856), que postuló que la razón de flujo de agua a través de un suelo sólido era proporcional al gradiente de la presión hidráulica. Los aparatos que se usaron en ese entonces hoy son conocidos como permeámetros de carga constante, como en la figura 3

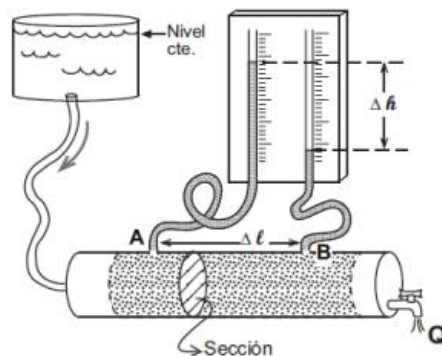


Figura 3: Permeámetro de carga constante (Sánchez, 2024).

Usando el esquema de la figura anterior, se tiene la Ecuación (3.1):

$$Q = K_s \cdot A \cdot \frac{\Delta h}{\Delta l} \quad (3.1)$$

Donde Q representa el caudal de salida, A es la sección transversal de flujo, Δh es la diferencia de altura entre las tomas piezométricas, Δl es la distancia entre ellas, $\Delta h/\Delta l$ es el gradiente de presión hidráulico y K_s es la conductividad hidráulica de saturación. En el experimento se establece que la razón de flujo de agua a través de un medio poroso está directamente regulada por la conductividad hidráulica, variable que es una representación de la facilidad de un fluido para pasar a través del lecho (Darcy, 1856, citado en Fredlund et al., 2012). Posteriormente, surge la necesidad de modelar el mismo problema observado por Darcy, pero para condiciones de saturación parcial del lecho. En 1907, Buckingham (1907) contribuye con su trabajo, postulando la siguiente ecuación, que describe la resistencia al flujo en suelos parcialmente saturados en estado estacionario:

$$\frac{Q}{A} = -K(h) \left(\frac{\partial h}{\partial z} + 1 \right) \quad (3.2)$$

Donde K es la conductividad hidráulica, y h es el potencial de succión. Como se puede ver en la ecuación (3.2), la conductividad hidráulica dependerá del potencial de succión, y a su vez dependerá de la humedad o grado de saturación del lecho, lo cual permite la generalización del fenómeno de escurrimiento en las condiciones enunciadas.

3.1.3. Ecuación de Richards

Richards (1931) logró describir la resistencia al flujo de un fluido en suelo parcialmente saturado en estado transiente, realizando un balance de materia al líquido en un volumen de control infinitesimal de un lecho de lixiviación, indicado en la Ecuación (3.3). Cabe destacar que en la dependencia espacial sólo se considera la distribución axial.

$$\frac{\partial \theta}{\partial t} = \frac{1}{A} \frac{\partial Q}{\partial z} \quad (3.3)$$

En donde θ es el contenido de humedad. Lo cual, agrupado con la Ecuación (3.2), resulta en:

$$\frac{Q}{A} = \frac{\partial}{\partial z} \left(K(h) \left(\frac{\partial h}{\partial z} - 1 \right) \right) \quad (3.4)$$

3.1.4. Conductividad hidráulica

Remontándonos a los inicios del estudio de la fluidodinámica en lechos porosos, Darcy (1856) definió la constante de proporcionalidad relacionada con el caudal, la sección transversal y el gradiente hidráulico como la conductividad hidráulica de saturación, también conocida como coeficiente de permeabilidad. Hemos observado que este valor varía según el potencial de succión y, por lo tanto, según el grado de saturación del lecho cuando se trata de suelos parcialmente saturados.

El agua solo puede fluir a través de la porción del medio poroso que contiene agua. Cuando la cantidad de agua en el suelo disminuye, la conductividad hidráulica también disminuye, ya que hay menos área transversal disponible para el flujo de agua. En el caso de la conductividad hidráulica relativa, que depende del contenido de agua, esta variable decrece a un ritmo mayor que el grado de saturación; más específicamente, disminuye logarítmicamente mientras el grado de saturación disminuye linealmente. (Fredlund et al., 2012).

También es importante notar que una reducción en la cantidad de agua en el suelo aumenta la tortuosidad, es decir, el camino real que recorre el fluido en comparación con la distancia directa. Además, la conductividad hidráulica presenta un comportamiento histerético similar al de la retención de líquido en un lecho, lo que significa que estos parámetros están directamente correlacionados con el historial de condiciones previas a las que ha sido sometido el lecho. Por ejemplo, la conductividad hidráulica será mayor en un lecho que pasa de un grado de saturación del 70% al 50%, que en el mismo lecho pasando del 30% al 50%, a pesar de que ambos casos tengan un grado de saturación equivalente. Esto explica la presencia de distintas curvas de retención bajo diferentes condiciones para un mismo lecho, como se muestra en la Figura 2. Además, se ha reportado que el uso de una tasa de riego escalonada contribuye a una mayor retención de líquido y, por ende, a una mayor conductividad hidráulica, en comparación con tasas de riego constantes, lo que acelera la formación del flujo de líquido. (Wang et al., 2021).

En lo que respecta a la conductividad hidráulica de saturación, la cual es propia de un lecho, se puede determinar utilizando un permeámetro de carga constante (replicando el experimento de Darcy), o mediante correlaciones que utilizan las características del suelo y del fluido a percolar, como son las correlaciones de Hazen (1892) y Kozeny-Carman (1997), determinación de la conductividad hidráulica de saturación escapa del alcance del trabajo, por lo que se utilizaran valores reportados en literatura.

Sin embargo, modelos de determinación de la conductividad hidráulica en lechos parcialmente saturados serán de gran importancia para la resolución del modelo a plantear. En primera instancia, van Genuchten (1980) derivó una ecuación que permite obtener analíticamente el comportamiento de las curvas de retención en base a la conductividad hidráulica $K(\theta)$ (Ecuación 3.5), en donde la expresión final contiene tres parámetros de ajuste que pueden ser obtenidos usando datos experimentales. Las Ecuaciones (3.5) y (3.6) corresponden a la familia paramétrica van Genuchten:

$$K(\theta) = K_s \sqrt{Se(\theta)} \left(\left(1 - Se^{\frac{1}{m}}(\theta) \right)^m \right)^2 \quad (3.5)$$

Donde $Se(\theta)$ es el grado de saturación, una forma adimensionalizada del contenido de humedad:

$$Se(\theta) = \frac{\theta(h) - \theta_r}{\theta_s - \theta_r} = \frac{1}{1 + (\alpha|h|^n)^m} \quad (3.6)$$

En estos casos, n y α son los parámetros de ajuste, siendo $m = 1 - 1/n$, mientras que θ_s es la humedad de saturación y θ_r es la humedad residual. El problema con esta definición yace en la presencia del valor absoluto del potencial de succión. La presencia de un valor absoluto presenta un problema a la hora de utilizar este modelo en un problema de programación matemática asociado a la toma de decisiones o supervisión de un proceso.

Para lograr los objetivos propuestos, es pertinente buscar modelos que describan la conductividad hidráulica que nos permita utilizarla en la simulación de este parámetro para poder entrenar nuestra red neuronal artificial. Es por esto, que en la Ecuación (3.7) se presenta el modelo descrito por Gardner (1958), el cual tiene forma exponencial. Considerando $K(\theta) = K_R \cdot K_S$:

$$K_R = \exp(\alpha h) = \left(\frac{\theta - \theta_r}{\theta_s - \theta_r} \right)^m \quad (3.7)$$

Este modelo tiene la ventaja de que, al tratarse de una exponencial, su derivación es sencilla, y junto con esto, Tracy (2006) comprobó que la curva de la Ecuación (3.7) en comparación al modelo de van Genuchten de la Ecuación (3.6) están lo suficientemente cercanas (Figura 4) para que el modelo exponencial cumpla con criterios de factibilidad física.

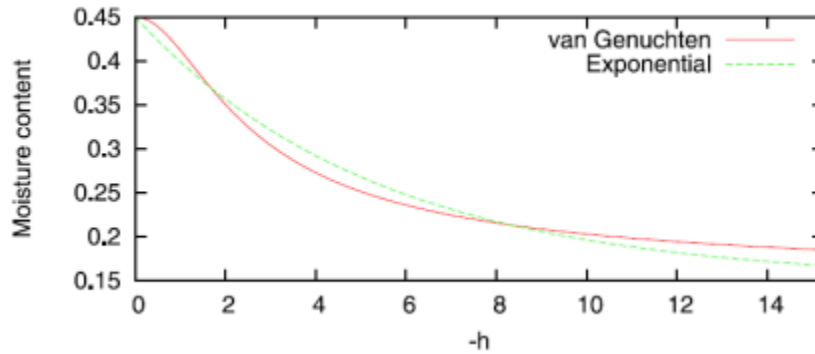


Figura 4: Contenido de humedad para modelos de van Genuchten y exponencial (Tracy, 2006).

3.2. Modelo Fenomenológico

Se cuenta con el modelo fenomenológico presentado por Olivares et al (2024), basado en la ecuación de Richards y las modificaciones de Bear y York (Bear, 1972) con sus condiciones de borde:

$$\frac{\partial \theta}{\partial t} = \frac{\partial}{\partial z} \left(D(\theta) \frac{\partial \theta}{\partial z} \right) - \frac{\partial}{\partial z} K(\theta) \quad (3.8a)$$

$$\theta = \theta_{ini}; \forall z \in [0, H] \quad (3.8b)$$

$$-D(\theta) \frac{\partial \theta}{\partial z} + K(\theta) = R(t); z = 0 \wedge t > t_0 \quad (3.8c)$$

$$\frac{\partial \theta}{\partial z} = 0; z = H \quad (3.8d)$$

Donde $D(\theta) = K(\theta) / (d\theta/dh)$ es el término difusivo, H la altura de la columna, θ la humedad relativa, z la coordenada axial, y $R(t)$ la tasa de riego dependiente del tiempo. Cabe destacar que, para resolver la ecuación, debemos determinar tres incógnitas: el término difusivo de la humedad, la conductividad hidráulica y la tasa de riego.

3.3. Machine Learning

Machine Learning es definido como la ciencia y arte de programar computadoras para que ellas puedan aprender de un set de datos (Gerón, 2019). Por ejemplo, un filtro de spam es un programa tipo Machine Learning (desde ahora como ML) el cual utiliza ejemplos de emails normales y emails spam ya etiquetados para poder diferenciar futuros emails entrantes al programa, proceso que tendrá un nivel de desempeño medible y que nosotros podemos controlar. Los programas tipo ML son especialmente útiles para las siguientes tareas:

- Problemas para los cuales las soluciones existentes requieren de mucho ajuste manual o una larga lista de reglas.
- Problemas complejos para los cuales no hay una buena solución usando métodos tradicionales; para estos los programas ML pueden encontrar soluciones.
- Entornos con variables fluctuantes en el tiempo, los programas ML pueden adaptarse a datos nuevos.
- Obtener nuevas perspectivas en problemas complejos de analizar o con gran cantidad de datos.

También existen diversas categorías para clasificar los programas de ML, si estos programas son entrenados con supervisión humana (supervisados, no supervisados o semi-supervisados), si estos

pueden aprender incrementalmente con datos suministrados en tiempo real (Online) o deben ser entrenados primero con todos los datos posibles antes de utilizarlo en un algoritmo (Batch), o si funcionan simplemente comparando puntos específicos en la data para predecir futuros puntos o detectan un patrón en la data y construyen un modelo predictivo (iteración en modelos fenomenológicos para encontrar parámetros). Para efectos de este trabajo, generalmente se considera que las redes neuronales artificiales o por sus siglas en inglés (ANN) pertenecen a la categoría de programa supervisado por humanos y dependiendo de cómo sean utilizadas en batch u online (Gerón, 2019).

Las principales complicaciones que tienen estos tipos de programas están principalmente relacionadas con los datos suministrados o el tipo de ajuste que se logra utilizando estos métodos. Primero, estos métodos requieren de cantidades grandes de datos para que puedan ser entrenadas y entreguen buenos resultados, dependiendo de la complejidad de tarea que tengan que realizar. Además, dependen del tipo de datos que se le suministre en el momento de su entrenamiento, ya que el ajuste se realizara para todos los datos, inclusive para datos no representativos o datos de baja calidad o con mucho ruido (Gerón, 2019).

3.4.Redes neuronales Artificiales

La Arquitectura de una Red Neuronal Artificial (ANN) consiste en una estructura organizada de neuronas artificiales interconectadas que trabajan conjuntamente para realizar tareas específicas, como clasificación, regresión, o reconocimiento de patrones. Las ANN se estructuran en capas (entrada, oculta/s, salida), donde cada capa tiene múltiples neuronas conectadas entre sí por sinapsis ponderadas. A continuación, se detallan los elementos fundamentales que conforman la arquitectura de una ANN.

3.4.1. Neuronas artificiales

Los principales *elementos* que constituyen a las ANN son las *Neuronas o nodos*, la *función de activación*, los *bias* y los *pesos*. Estos elementos se explicarán a continuación:

Neuronas o Nodos: Las unidades básicas de procesamiento en una ANN, estas son inspiradas y modeladas a partir de las neuronas biológicas. Como se puede ver en la figura 5, cada neurona recibe una serie de entradas, las combina, aplica una función de activación y produce una salida. Una neurona individual en una red neuronal puede conceptualizarse mediante la siguiente ecuación:

$$y = f\left(\sum_{i=1}^n w_i x_i + b\right) \quad (3.9)$$

Donde y es la salida de la neurona, f es la función de activación, w_i son los pesos asociados a cada entrada (x_i), x_i son las entradas a la neurona, y b es el término de sesgo (bias).

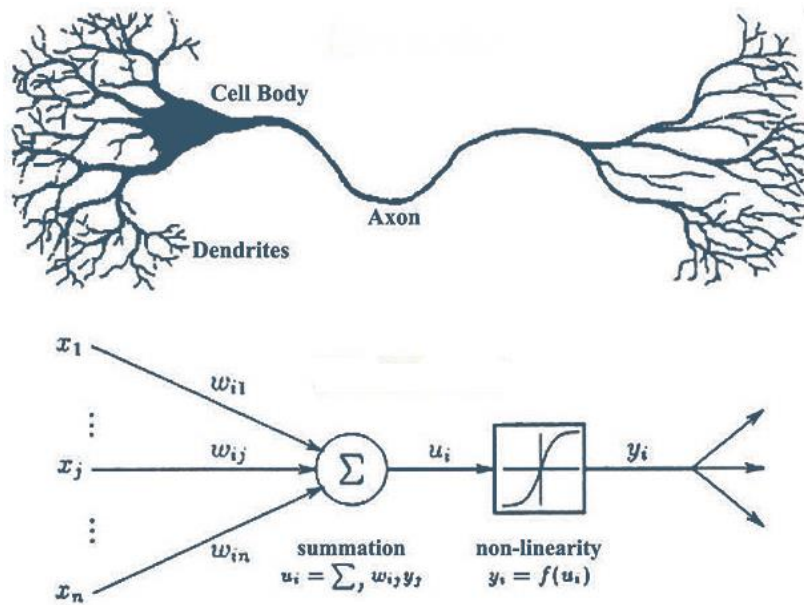


Figura 5: Diagrama Red neuronal Artificial. (Gerón, 2019).

Función de Activación: esta función se aplica después de la suma del bias y los pesos a la entrada de la neurona produciendo una salida. La función de activación (f) introduce no linealidad al modelo, permitiendo a la red aprender patrones complejos (Gerón, 2019). Funciones de activación comunes incluyen la función sigmoide, ReLU (Rectified Linear Unit), tangente hiperbólica ($\tanh$), y softmax. La elección de la función de activación afecta la capacidad de la red para aprender patrones complejos y dependerá del tipo de resultados que se busque obtener para una determinada tarea. Algunas funciones de activación comunes incluyen:

- Sigmoide: $[f(x) = \frac{1}{1+e^{-x}}]$

- ReLU (Rectified Linear Unit): $[f(x) = \max(0, x)]$

- Tanh (Tangente hiperbólica): $[f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}]$

- Softmax (para la capa de salida en clasificación multiclase): $[\sigma(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}]$

Bias: Término que se aplica a las neuronas de cada capa (excepto la capa de entrada) para mejorar la flexibilidad del modelo. El bias permite que la función de activación se desplace hacia la izquierda o la derecha, lo que ayuda a la red a aprender patrones más complejos.

Pesos: Factor que multiplica las entradas en las conexiones. El ajuste de estos pesos durante el entrenamiento es fundamental para que la red aprenda a realizar su tarea.

3.4.2. Capas

A continuación, se describen los diferentes tipos de capas encontrados dentro de las ANN y que conforman la estructura de estas:

Capa de Entrada: Recibe los datos de entrada del modelo. Cada neurona en esta capa representa una característica de los datos de entrada.

Capas Ocultas: Intermedias entre la entrada y la salida, donde se realiza el procesamiento y la transformación de los datos. Una ANN puede tener varias capas ocultas, y cada una puede tener un número variable de neuronas.

Capa de Salida: Produce la salida final del modelo. El número de neuronas en esta capa depende de la tarea específica (por ejemplo, clasificación binaria, multiclase, regresión).

En una red con múltiples capas, la salida de una capa se convierte en la entrada de la siguiente. La ecuación para una capa oculta o la capa de salida se generaliza como:

$$[y = f(Wx + b)] \quad (3.10)$$

Donde: y es el vector de salidas de la capa, W es la matriz de pesos que conecta la capa anterior con la actual, x es el vector de entradas de la capa (o la salida de la capa anterior), b es el vector de sesgo de la capa, f es la función de activación aplicada elemento a elemento.

3.4.3. Función de pérdida

Una función de pérdida o de coste evalúa qué tan bien la ANN está realizando su tarea, comparando las salidas predichas con las verdaderas. Ejemplos comunes incluyen el error cuadrático medio (MSE) para regresión y la entropía cruzada para clasificación. La función de pérdida cuantifica el error entre las predicciones de la red y los valores reales. Algunas funciones de pérdida comunes son:

- Error Cuadrático Medio (para regresión): $[L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2]$
- Entropía Cruzada (para clasificación): $[L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})]$

Donde: N es el número de muestras, C es el número de clases, y_i es el valor real, y $\hat{y}_i$ es la predicción de la red.

Obtenido un valor de pérdida, ahora es necesario que la red realice un aprendizaje automático, esto consiste en ajustar los pesos sinápticos de la red para minimizar la diferencia entre las predicciones de la red y los valores reales que son el objetivo de entrenamiento de la red.

Para ajustar los pesos, se utiliza un algoritmo que modifica los pesos de las conexiones en la red con el objetivo de minimizar la función de pérdida. Ejemplos de algoritmos optimizadores incluyen el Descenso del Gradiente Estocástico (SGD), Adam, LBFGS-N, y RMSprop.

La optimización de estos pesos, típicamente realizada a través del algoritmo de retropropagación y un algoritmo como el Descenso del Gradiente, ajusta los pesos para minimizar la función de pérdida. La actualización de pesos se realiza como sigue:

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w} \quad (3.11)$$

Donde:

- (w_{new}) y (w_{old}) son los pesos después y antes de la actualización, respectivamente,
- (η) es la tasa de aprendizaje, y
- ($\frac{\partial L}{\partial w}$) es el gradiente de la función de pérdida respecto al peso.

Estas ecuaciones forman la base de la mayoría de las ANN, desde las más simples hasta las más complejas, incluidas las variantes como las redes neuronales convolucionales (CNN) y las redes neuronales recurrentes (RNN), que adaptan y expanden estos principios básicos para tareas específicas.

Adicionalmente existen otros hiperparámetros que controlan otras características del entrenamiento de las ANN, como lo son la *tasa de aprendizaje* y *numero de épocas de entrenamiento*. Estos hiperparámetros se detallan a continuación:

Tasa de aprendizaje: Un hiperparámetro que controla cuánto ajustamos los pesos de nuestra red con respecto al gradiente de la función de pérdida. Una tasa de aprendizaje demasiado alta puede hacer que el aprendizaje sea inestable, mientras que una tasa demasiado baja puede hacer que el aprendizaje sea muy lento (Gerón, 2019).

Numero de épocas de entrenamiento: Hiperparámetro que controla la cantidad de ciclos máximos que el optimizador buscará reducir el valor de la función de pérdida. En general es recomendable terminar con el entrenamiento cuando otra variable llegue a un valor límite, como lo puede ser el gradiente de la función de pérdida respecto al peso, ya que aumentar los ciclos de entrenamiento puede generar un sobre ajuste en la red. Sin embargo, se deben dar los ciclos necesarios para que los valores de gradiente de función de pérdida y valor de pérdida lleguen a un mínimo.

3.4.4. Arquitectura Compleja

La complejidad de una ANN, determinada por el número de capas ocultas y el número de neuronas en cada capa, permite a la red aprender patrones cada vez más complejos. Sin embargo, redes más grandes también son más propensas al sobreajuste y pueden requerir más datos y tiempo de entrenamiento. La arquitectura de una ANN debe diseñarse cuidadosamente para equilibrar la capacidad de la red para aprender patrones complejos con el riesgo de sobreajuste y la eficiencia computacional.

3.4.5. Redes neuronales basadas en procesos físicos (PINN)

Las PINN son una técnica de implementación de las redes neuronales artificiales, las cuales para circuncidar las desventajas de los métodos basados solo en datos como las ANN, incorporan dentro de la función de pérdida información de las ecuaciones físicas que gobiernan el problema físico, sus condiciones de borde y condiciones iniciales. Entonces con la incorporación de conocimiento previo conocimiento físico, las PINN presentan resultados más confiables con menos datos alimentados al sistema, lo que las hace un tipo de red neuronal más apropiada para ciertos problemas físicos. (Depina et al, 2022)

4. METODOLOGÍA

La metodología utilizada en el presente trabajo involucró una primera etapa donde se determinaron los hiperpárametros y métricas de validación para las redes neuronales a integrar, seguido de una segunda etapa de implementación de las redes neuronales artificiales con el fin de lograr la descripción espacial y temporal de la humedad en una pila de lixiviación.

4.1. Determinación de hiperpárametros de las Redes neuronales

Se propuso la siguiente metodología de trabajo para integrar las redes neuronales artificiales o ANN al modelo fenomenológico de Richards.

Con el fin de cerrar los grados de libertad del problema y considerando que la tasa de riego es la variable por manipular, se utilizó una red neuronal profunda (DNN) entrenada con datos de conductividad hidráulica K y difusividad hidráulica D como se muestra en las ecuaciones (4.1) y (4.2).

Esto con la finalidad de predecir los datos de humedad producidos con el modelo de Richards. Esto permitirá describir el perfil espaciotemporal de la humedad dentro de una pila de lixiviación.

$$K(\theta) = DNN(\theta(h)) \rightarrow K(\theta) = K_s \sqrt{Se(\theta)} \left(\left(1 - Se(\theta)^{\frac{1}{m}} \right)^m \right)^2 \quad (4.1)$$

$$D(\theta) = DNN(\theta(h)) \quad (4.2)$$

Donde, $K(\theta)$ es la conductividad hidráulica, $D(\theta)$ es la difusividad hidráulica en el medio, $Se(\theta)$ es la humedad relativa que es función de la humedad θ , m es un parámetro de ajuste de la ecuación de van Genuchten y K_s es la conductividad hidráulica de saturación. Los datos utilizados para el entrenamiento de estas redes neuronales constaran de datos simulados utilizando las ecuaciones de curva de retención de van Genuchten.

Para la primera etapa, se comenzó simulando el modelo fenomenológico suponiendo una curva de retención conocida con el fin de obtener datos y así cumplir con dos hitos importantes de esta etapa: aprender a construir una red neuronal profunda (DNN) implementada en Matlab y determinar los hiperpárametros más relevantes que conforman esta red neuronal profunda con tal de que esta sea capaz de predecir las variables previamente indicadas.

El beneficio de simular los datos radica en dos aspectos principales: primero, suplir la falta de datos reales debido a la dificultad actual para obtener mediciones de humedad dentro de una columna; y segundo, generalizar los resultados obtenidos.

Entonces para obtener los datos simulados necesarios, se propone el modelo matemático comprendido dentro del siguiente dominio.

En la figura 6 se representa una columna de largo H , humectada con una razón de regado $R(t)$. En la base de la columna, el flujo de salida q_0 es medido en intervalos de tiempo específicos t_l , $l \in 1, \dots, N$. Si $\theta = \theta(z, t)$ es el contenido de humedad de la columna a una altura $z \in [0, H]$ a un tiempo $t \in [0, T]$, de acuerdo con la ley de Darcy y la ecuación de Richards (Bear, 1972) se obtiene el modelo central y sus condiciones de borde, previamente descritas en las ecuaciones 3.8:

$$\frac{\partial \theta}{\partial t} = \frac{\partial}{\partial z} \left(D(\theta) \frac{\partial \theta}{\partial z} \right) - \frac{\partial}{\partial z} K(\theta) \quad (3.8a)$$

$$\theta = \theta_{ini}; \forall z \in [0, H] \quad (3.8b)$$

$$-D(\theta) \frac{\partial \theta}{\partial z} + K(\theta) = R(t); z = 0 \wedge t > t_0 \quad (3.8c)$$

$$\frac{\partial \theta}{\partial z} = 0; z = H \quad (3.8d)$$

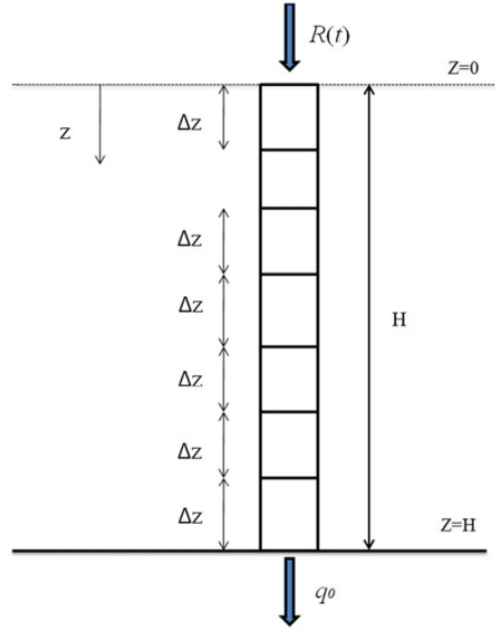


Figura 6: Dominio matemático del modelo (Cariaga et al, 2015)

El algoritmo numérico que resuelve la ecuación de Richards dentro de este dominio es el método de discretización de Crank-Nicolson detallado en el anexo C, presentado por Cariaga et al. (2015), en conjunto con las ecuaciones de Van Genuchten presentadas a continuación:

$$K(\theta) = K_s \sqrt{Se(\theta)} \left(\left(1 - Se(\theta)^{\frac{1}{m}} \right)^m \right)^2 \quad (4.3a)$$

$$Se(\theta) = \frac{\theta(h) - \theta_r}{\theta_s - \theta_r} = \frac{1}{1 + (\alpha|h|^n)^m} \quad (4.3b)$$

$$\frac{d\theta}{dh} = c * \frac{(\alpha * h)^{n-1}}{(1 + (\alpha * h)^n)^{m+1}} ; c = (\theta_s - \theta_r) * n * \alpha * (-m) \quad (4.3c)$$

$$D(\theta) = \frac{K(\theta)}{\frac{d\theta}{dh}} \quad (4.3d)$$

Donde, $K(\theta)$ es la conductividad hidráulica, $D(\theta)$ es la difusión en el medio, $Se(\theta)$ es la humedad relativa, θ_s es la humedad de saturación del lecho, θ_r es la humedad residual del lecho, h es la presión de succión, K_s es la conductividad hidráulica de saturación del lecho y n , m y α son parámetros de Van Genuchten.

Los datos son simulados utilizando el método matemático discretizado de Crank-Nicolson y para obtener las variables $K(\theta)$ y $D(\theta)$. Se utilizó las ecuaciones y parámetros de Van Genuchten. Los parámetros necesarios para su simulación se encuentran en la tabla 1 en el anexo A, obtenidos de Cariaga et al. (2015). El software utilizado para simular los datos con la curva de retención fue Matlab.

Una vez simulados los datos de $Se(\theta)$, $K(\theta)$ y $D(\theta)$, se construye las redes neuronales utilizando el software Matlab junto con la extensión DeepNetwork Toolbox. Se construyeron dos redes neuronales, cada una con objetivo de realizar una regresión sobre las variables de $K(\theta)$ y $D(\theta)$. Para esto cada red cuenta con una serie de elementos propuestos a continuación:

1. Una neurona de entrada, debido a que las variables a predecir $K(\theta)$ y $D(\theta)$ dependen de solo de $Se(\theta)$.
2. Una serie de subcapas, determinadas por los hiperparámetros numero de capas y numero de neuronas por capas.
3. la función de activación de cada neurona, tangente hiperbólica debido a que la variable a predecir es continua.
4. Una neurona de salida, que entrega la variable a predecir.

Con los elementos previamente mencionados, la arquitectura propuesta fue de tipo Feed-forward, es decir, que procesa la información en una sola dirección, esto debido a que, al tratarse de un problema de regresión, esta es la arquitectura recomendada para una red simple de implementar y efectiva en problemas de este tipo (Gerón, 2019).

También, en esta etapa del trabajo se realizó una sensibilización enfocada al número de capas y número de neuronas por capa con el fin de determinar el número mínimo de estos parámetros que genere un ajuste satisfactorio de las variables $K(\theta)$ y $D(\theta)$, esto debido a que se consideró que

estos hiperparámetros son los más relevantes para el caso de estudio ya que tienen una consecuencia directa sobre la convergencia de las redes sobre los datos suministrados.

El código utilizado para la generación de datos y, construir y entrenar estas redes neuronales profundas enfocadas a la sensibilización de los hiperpárametros se encuentra en el anexo H. Cabe mencionar que, en la segunda parte de la integración de las redes neuronales profundas en el modelo matemático, se evaluó el efecto de la arquitectura de las redes en la predicción del perfil de humedad.

4.2. Estrategia de entrenamiento e incorporación al modelo

Entonces para poder integrar las redes neuronales se propone el siguiente esquema de flujo de información.

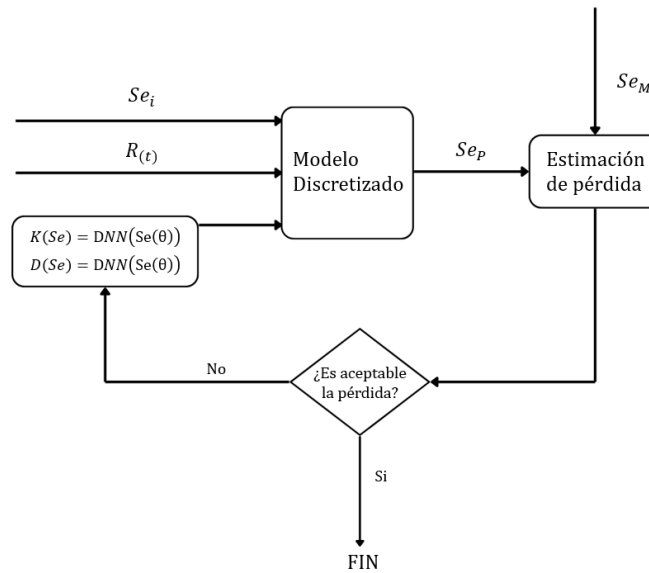


Figura 7: Diagrama de flujo de incorporación del modelo (elaboración propia)

De acuerdo con la figura 7, para incorporar las redes neuronales artificiales, es necesario contar con un modelo discretizado que represente el problema bajo estudio. Este modelo fenomenológico, presentado por Cariaga et al (2015), se basa en la ecuación de Richards, junto con las modificaciones propuestas por Bear y York (Bear, 1972), y utiliza diferencias finitas de Crank-Nicolson como método de discretización temporal y espacial.

La estrategia comienza con la alimentación al modelo de un valor inicial de Se , que corresponde a la humedad relativa adimensionalizada, y un $R(t)$, que representa la tasa de riego por cada día de simulación. A diferencia de trabajos anteriores, no se asume una estructura predefinida para las curvas de retención. En su lugar, se utilizan redes neuronales para predecir los valores de difusividad y conductividad hidráulica ($D(\theta)$ y $K(\theta)$, respectivamente).

Con estos valores, se simula el perfil de humedades relativas adimensionalizadas y se estima un error comparando los resultados con valores de Se "medidos" o considerados como referencia dentro del rango de la simulación.

Si las diferencias entre los valores predichos por las redes neuronales y los valores medidos son aceptables, se obtienen redes capaces de simular el comportamiento de las variables medidas. En caso contrario, si las diferencias no son satisfactorias, las redes aún no son capaces de predecir los valores deseados, por lo que es necesario actualizar sus subparámetros (pesos y sesgos) hasta que la pérdida entre los valores predichos y los objetivos sea aceptable.

Además de esto y como se mencionó en el punto anterior, para la integración en general de las redes neuronales profundas en el modelo matemático, se evaluó el efecto de la arquitectura de las redes en la predicción del perfil de humedad manteniendo el modelo matemático constante.

Para esto se seleccionaron dos tipos de arquitectura, en la primera se construyó una red con una entrada correspondiente a la humedad relativa (Se), para el cuerpo de la red se utilizaron un número de capas y número de neuronas por capas determinadas en la primera parte, cada una conectada a una función de activación tangente hiperbólica, por último, para las variables a predecir se utilizaron dos neuronas de salida, una para la difusividad hidráulica $D(Se)$ y otra para la conductividad hidráulica $K(Se)$.

En la segunda arquitectura, se mantienen los elementos de la entrada y cuerpo de la red constantes, diferenciándose en que para las variables a predecir solo hay una neurona de salida, es decir, se deben construir dos redes artificiales por separado, una para predecir los valores de $D(Se)$ y otra para los valores de $K(Se)$.

Entonces para implementar la estrategia descrita anteriormente, se comenzó por el entrenamiento de estas redes utilizando datos obtenidos de la misma manera que en la primera parte de la implementación, con el modelo de Crank - Nicolson en conjunto con las curvas de retención de Van Genuchten. Para el entrenamiento en sí, se propuso el siguiente algoritmo de la figura 8, el código de dicho algoritmo fue desarrollado en Matlab y se encuentra en el anexo I:

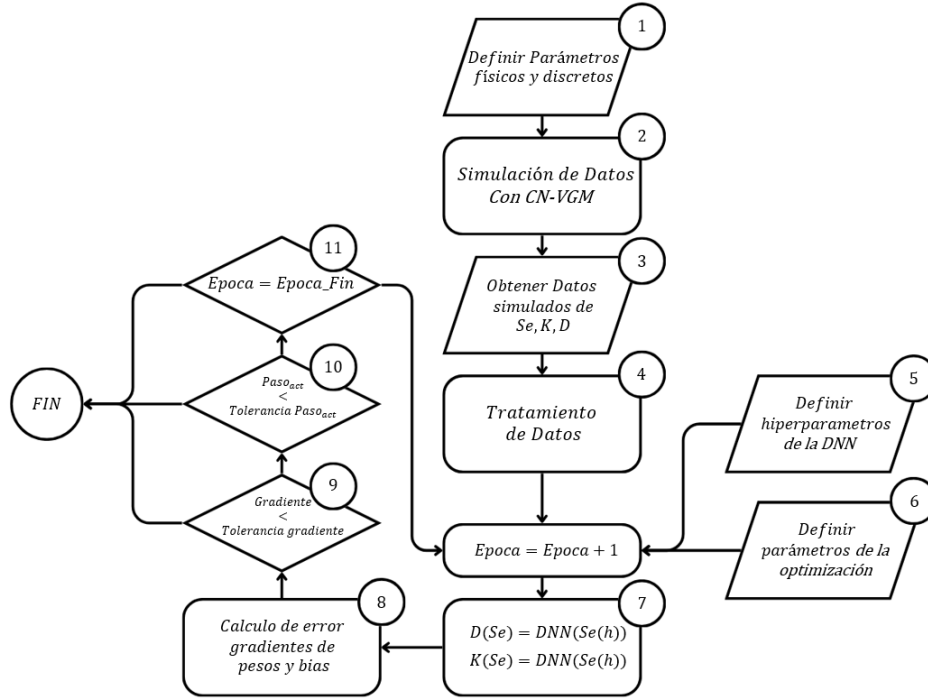


Figura 8: Algoritmo de entrenamiento DNN

En este algoritmo, se definen por separado los parámetros físicos y de discretización utilizados (Paso 1), los parámetros del algoritmo de optimización (Paso 6), y los hiperparámetros y la arquitectura de la red a entrenar (Paso 5). Todos estos parámetros se detallan en el anexo A. Una vez establecidos, se simulan los valores de Se , D y K utilizando los parámetros físicos y discretos (Paso 2). Estos valores obtenidos se convierten en la entrada para el entrenamiento (Paso 3).

Antes de entrenar las redes con estos datos, es necesario realizar un tratamiento previo a los datos de entrada (Paso 4) para facilitar la predicción. Esto se debe a que la función de activación de las redes, la tangente hiperbólica, genera valores en el dominio $[-1,1]$, por lo que es conveniente normalizar los datos de D y K para reducir la complejidad matemática y mejorar la capacidad predictiva de la red. Para normalizar, se divide cada conjunto de datos por su valor máximo, tanto para K como para D . Además, los datos disponibles se dividen en conjuntos de entrenamiento y validación, lo que permite detectar posibles sobreajustes de la red en caso de que ocurran.

Una vez realizado el tratamiento, comienza el entrenamiento de la red (a partir del Paso 4). En la primera época, se inicializan las neuronas y se evalúa la función de pérdida. En este proceso, primero se predicen los valores de D y K (D_{pred} , K_{pred}) utilizando la humedad (Se) como dato de entrada (Paso 7).

Luego, se calcula la norma L_2 de los residuos entre las predicciones (D_{pred} , K_{pred}) y los valores objetivo (D_0 , K_0). Con esta medida, se calcula el gradiente de los pesos y sesgos en relación con el error (Paso 8).

Finalmente, el algoritmo de actualización Limited-memory Broyden-Fletcher-Goldfarb-Shanno (L-BFGS) predice el siguiente paso de actualización utilizando los gradientes calculados (Paso 8).

Para que el algoritmo continúe actualizando, deben cumplirse tres condiciones:

1. El gradiente no debe ser menor que el parámetro de tolerancia de gradiente (Paso 9), evitando así la implosión de gradientes en las redes.
2. El paso de actualización no debe ser menor que el parámetro de tolerancia del paso de actualización (Paso 10), lo que previene cambios demasiado pequeños en la red.
3. La época actual debe ser menor que el número máximo de épocas definido (Paso 11), que marca el fin del proceso de optimización.

Este proceso se repite hasta que alguna de las condiciones de los pasos 9, 10 o 11 se cumpla.

Para el algoritmo, los parámetros utilizados para la obtención de datos se encuentran en anexo A, los hiperparámetros para las redes fueron previamente definidos en la primera parte de la implementación y los parámetros de optimización se encuentran en el código del anexo I.

Una vez finalizado el entrenamiento, las redes neuronales entrenadas se integran en el modelo de Crank-Nicolson, reemplazando las ecuaciones de Van Genuchten modificado. Esto permite obtener, en cada caso, un perfil de humedades predicho utilizando las redes entrenadas. El código desarrollado en Matlab para esta incorporación se encuentra en el anexo J. A continuación, se presenta la figura 9 que ilustra la comparación entre el modelo de Crank-Nicolson con Van Genuchten y el modelo con redes neuronales.

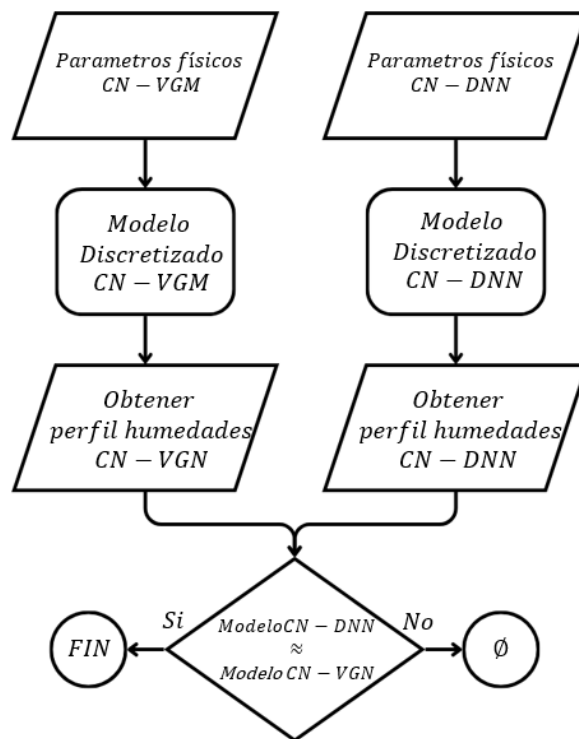


Figura 9: Comparación entre modelo con VGM y modelo con DNN (elaboración propia)

En este algoritmo compara los modelos con Van Genuchten Modificado (VGM) y con redes neuronales artificiales profundas (DNN).

Para comparar, primero se empieza por definir los parámetros necesarios para cada tipo de modelo, luego en paralelo se simula el perfil de humedades adimensionales y por último se compara si los modelos entregan comportamientos similares para los valores de las humedades.

De no ser comparables tenemos un modelo con un conjunto de parámetros insuficientes para la simulación matemática, representado por un conjunto vacío ($\emptyset$), por lo que es necesario revisar la implementación y/o validación del entrenamiento y realizar cambios hasta que los modelos entreguen comportamientos similares.

Por último, cabe mencionar que la validación de las redes neuronales obtenidas se realiza después del entrenamiento de estas. Para esto se utilizan las siguientes gráficas y métricas:

- Gráfico de pérdida de entrenamiento vs validación, para detectar posibles sobreajustes producidos.
- Métrica R^2 para medir el rendimiento de la red. Histograma de error, para analizar la distribución de los valores residuales.
- Diagrama de dispersión, para comparar visualmente las predicciones con los valores reales.

Para la comparación entre los perfiles de humedad se seleccionaron el valor R^2 y el diagrama de dispersión para determinar el rendimiento del modelo con redes neuronales respecto al modelo sin redes.

5. RESULTADOS Y DISCUSIÓN

5.1. Determinación de Hiperpárametros de las redes neuronales

A continuación, se muestran gráficos de dispersión de la red neuronal obtenidas en la figura 10 y los gráficos de predicción en la figura 11, las características utilizadas para las redes neuronales entrenadas son las mencionadas en el apartado 4.1 y el algoritmo de optimización utilizado fue el Limited-memory Broyden-Fletcher-Goldfarb-Shanno (L-BFGS), a esta red le fueron alimentados 1057 datos diferentes.:

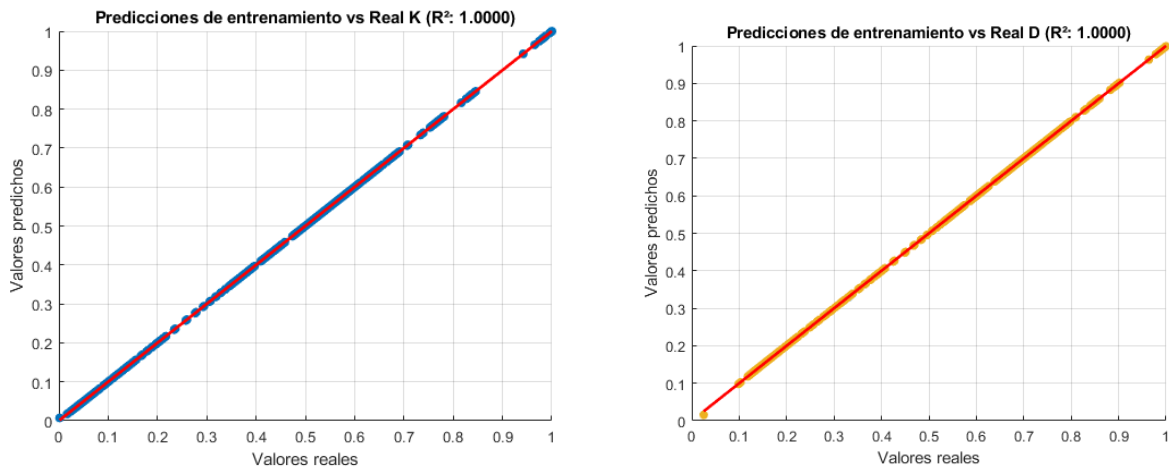


Figura 10: Gráficos de dispersión para datos de entrenamiento

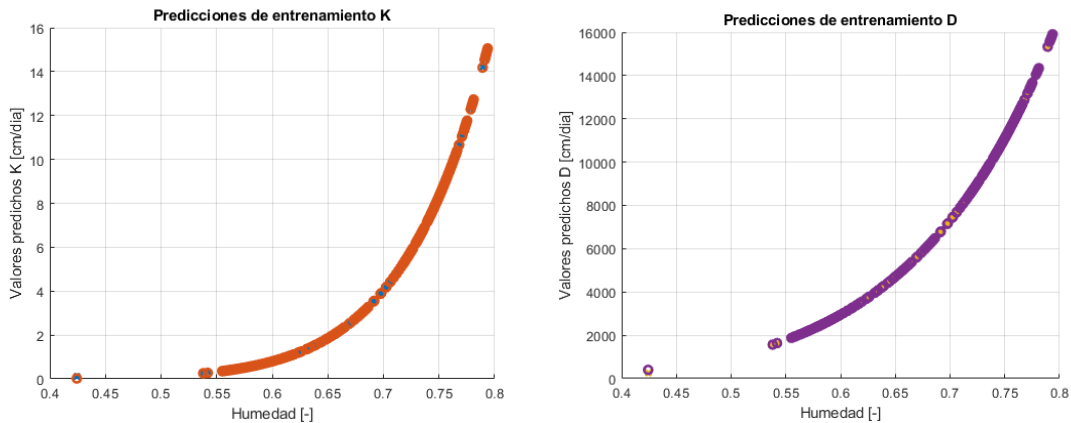


Figura 11 :Gráficos de predicción de K y D respectivamente.

La figura 10 muestra la relación entre las salidas de la red neuronal y los valores objetivos, específicamente entre los valores de $K(\theta)$ predichos por el modelo y los valores provenientes de los datos alimentados al modelo. En estos gráficos se observa un buen ajuste, ya que el término R^2 , que representa el ajuste del modelo respecto de los datos utilizados, es igual a 1.

Por otro lado, la figura 11 presenta las predicciones realizadas por las redes neuronales, representadas con círculos naranjos para K y morados para D, junto con los datos objetivos, indicados con equis azules. En la tabla de resultados 3 y 4 de sensibilización para K y D, ubicada en el anexo B, se detalla el número de capas, el número de neuronas por capa y el error normalizado entre las predicciones y los datos de entrada. El criterio de error utilizado para determinar el número mínimo de capas y neuronas fue un error normalizado de 0.001.

El último valor de las tablas 3 y 4 en el anexo B corresponde a la configuración mínima de capas y neuronas por capa que cumplen con este criterio. Para la conductividad hidráulica K, se encontró que con 1 capa y 7 neuronas se obtiene un ajuste satisfactorio para este conjunto de datos en particular. Para D, se determinó que 1 capa y 10 neuronas son suficientes para lograr un ajuste adecuado.

Esto indica que la complejidad necesaria de las redes neuronales para predecir estos datos no es alta, aun cuando la relación entre los datos de entrada y los datos de salida es no lineal.

Cabe destacar que los valores encontrados son una aproximación al valor óptimo de número de capas y número de neuronas por capas, ya que un aumento en el número de capas y neuronas podría reducir aún más el error normalizado, mejorando la precisión y generalización de la red, aunque esto también conlleva un mayor riesgo de sobreajuste en las redes y posibilidad de errores en el entrenamiento como la explosión de gradientes.

A pesar de esto, los valores encontrados por este método son una buena aproximación al punto de partida de los hiperparámetros de las redes neuronales a implementar.

5.2. Resultados del entrenamiento e integración de redes neuronales

Los resultados obtenidos del entrenamiento y la implementación se encuentran a continuación. Es importante destacar que las métricas y gráficos se organizan según la etapa en la que fueron generados. Los resultados se clasifican en dos categorías:

- **Validación:** Corresponde a los resultados obtenidos durante el entrenamiento de las redes, tanto para la red con dos salidas como para el conjunto de redes que cuentan con una salida por cada variable.
- **Comparación entre métodos:** Incluye los resultados asociados a los perfiles de humedad generados por el método de Crank-Nicolson con Van Genuchten modificado (utilizado como caso base), el método de Crank-Nicolson con la red de dos salidas, y el método de Crank-Nicolson con las redes de una salida por cada variable.

A partir de este punto, la red neuronal profunda con dos salidas se denominará Red_J , mientras que el par de redes con una salida por variable se denominarán Red_K (para la conductividad hidráulica) y Red_d (para la difusividad hidráulica), respectivamente.

5.2.1. Validación redes neuronales

En primer lugar, se presentan los resultados principales de la validación de la red neuronal con dos salidas, Red_J .

Es importante destacar que, para el entrenamiento de todas las redes, se utilizaron datos simulados correspondientes al primer instante de posición de la simulación matemática con curvas de retención y método de Crank-Nicolson, lo que representa un total de 1057 datos de humedad relativa adimensionalizada, en un rango de $[0.4242, \dots, 0.7931]$. Esta elección se debe a que se busca reducir la cantidad de datos necesarios para entrenar la red neuronal, incrementando así la factibilidad física de reproducir el experimento con datos reales.



Figura 12: Desempeño de entrenamiento y validación Red_J .

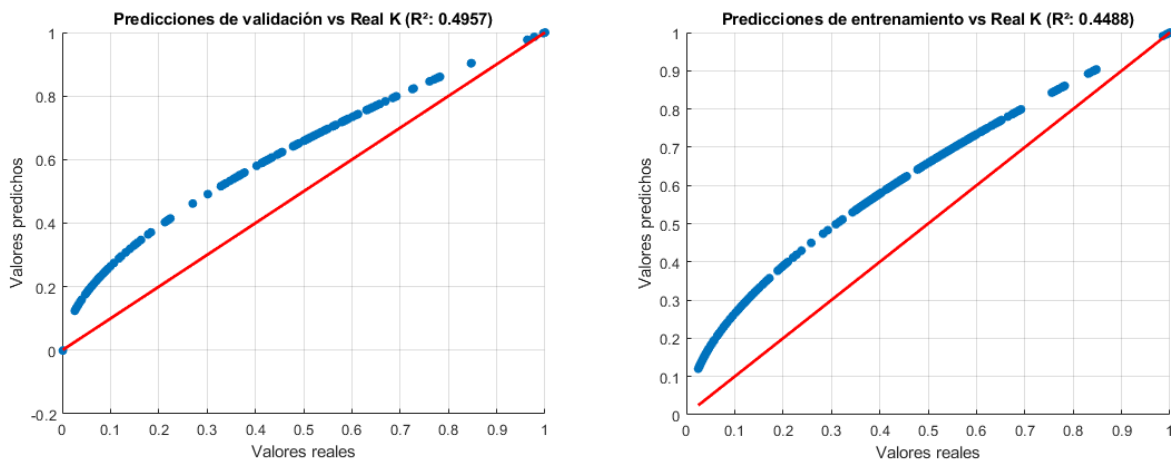


Figura 13: Gráficos de dispersión de la Red_J para la variable K, el gráfico de la izquierda corresponde a los datos de validación y el de la derecha corresponde a los datos de entrenamiento.

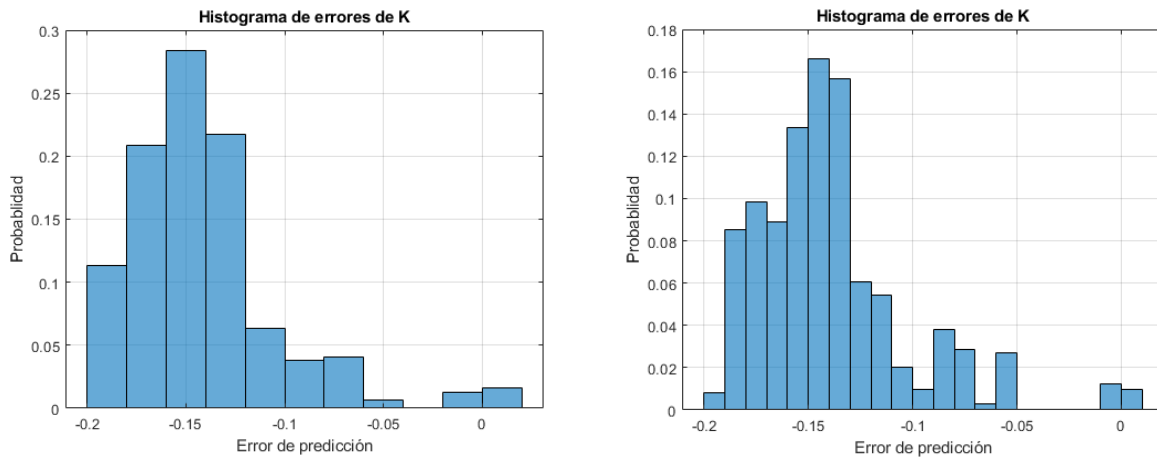


Figura 14: Histogramas de error de Red_j para la variable K, el histograma de la izquierda corresponde a los datos de validación y el histograma de la derecha corresponde a los datos de entrenamiento.

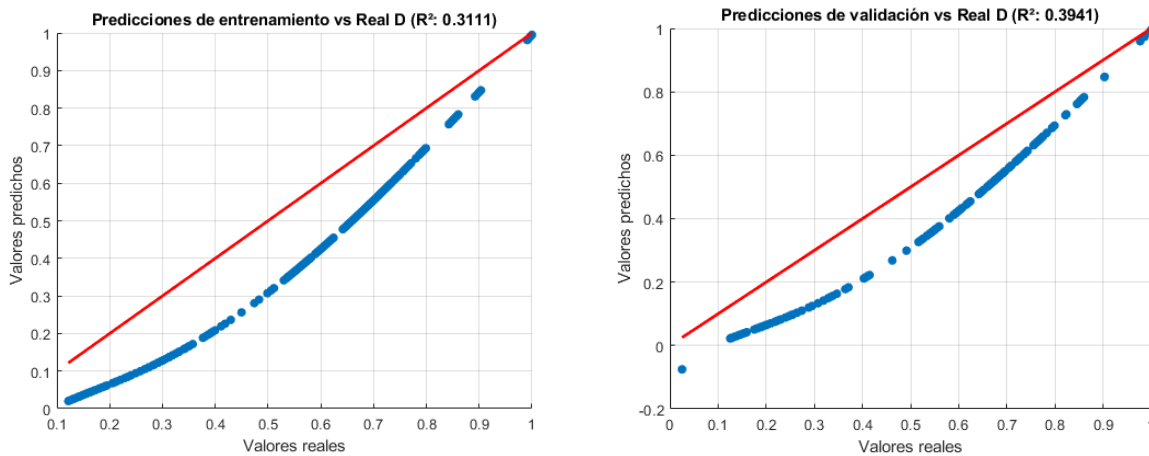


Figura 15: Gráficos de dispersión de la Red_j para la variable D, el grafico de la izquierda corresponde a los datos de validación y el de la derecha corresponde a los datos de entrenamiento.

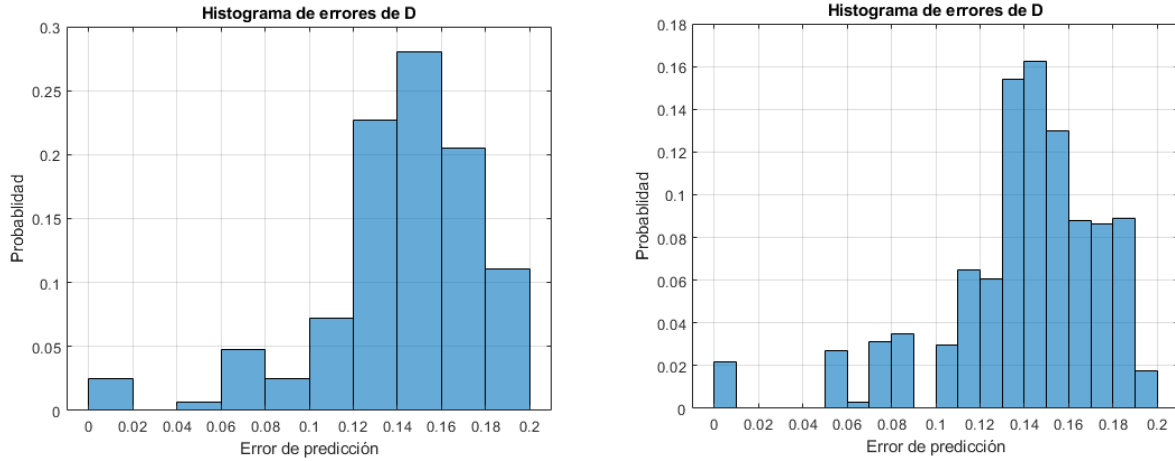


Figura 16: Histogramas de error de Red_j para la variable D, el histograma de la izquierda corresponde a los datos de validación y el histograma de la derecha corresponde a los datos de entrenamiento.

De la figura 12 se observa que la línea que representa el desempeño de la red con los datos de validación se encuentra por encima de la línea correspondiente a los datos de entrenamiento. Esto indica que la red está sobrestimando los datos, lo que resulta en una mala capacidad de generalización al momento de predecir con entradas diferentes a las utilizadas durante el entrenamiento. Esta situación se refleja en la diferencia entre los valores de R^2 obtenidos para los datos de entrenamiento y los de validación.

Además, se observa que los valores de R^2 de los gráficos de dispersión en la figura 13 y 15 son inferiores a 0.5, tanto para la conductividad hidráulica (K) como para la difusividad hidráulica (D), tanto para los datos de entrenamiento como para los datos de validación, lo que sugiere que el modelo es poco confiable para predecir estos parámetros. Esto podría deberse a que la arquitectura de la red no es la más adecuada para este conjunto de datos.

Por otro lado, los histogramas de las figuras 14 y 16 muestran que la mayoría de los errores se concentran alrededor de -0.15 para el caso de K y 0.15 para el caso de D, lo que sugiere la presencia de un sesgo sistemático en la red. Este sesgo podría estar relacionado con la arquitectura utilizada, ya que al requerir dos salidas diferentes a partir de una misma entrada, las interacciones entre las neuronas podrían estar generando imprecisiones en las respuestas obtenidas por la red Red_j . Sin embargo, los errores de las predicciones poseen una baja varianza, lo que indica una alta precisión en la predicción de valores. Por último, comparando los conjuntos de los histogramas, se puede ver una similitud en la forma de la distribución, lo que indica una buena capacidad de generalización de la red.

A continuación, se presentan los resultados principales para la validación del par de redes neuronales Red_k . (para la conductividad hidráulica) y Red_d . (para la difusividad hidráulica), cada una con una salida para las variables D y K respectivamente.

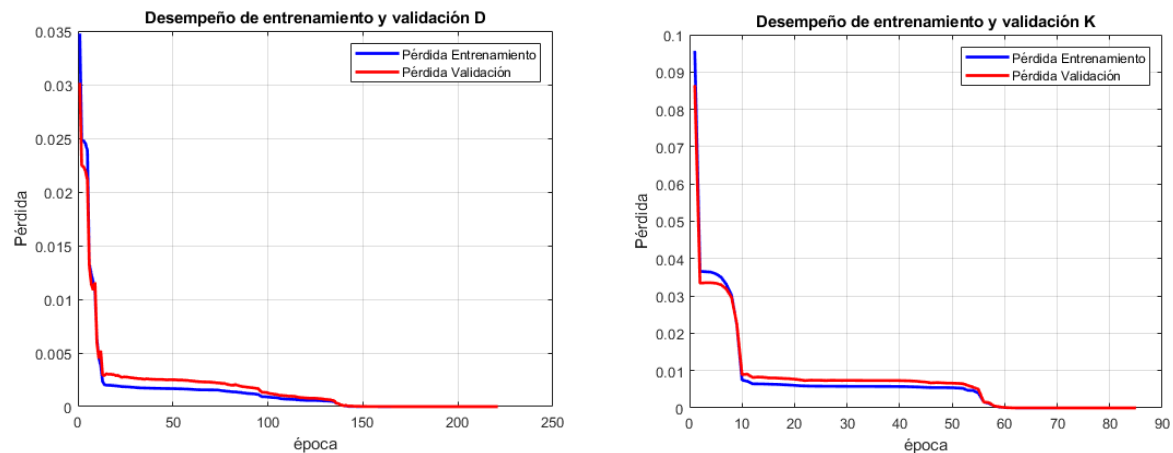


Figura 17: Desempeño de entrenamiento y validación Red_d y Red_k respectivamente.

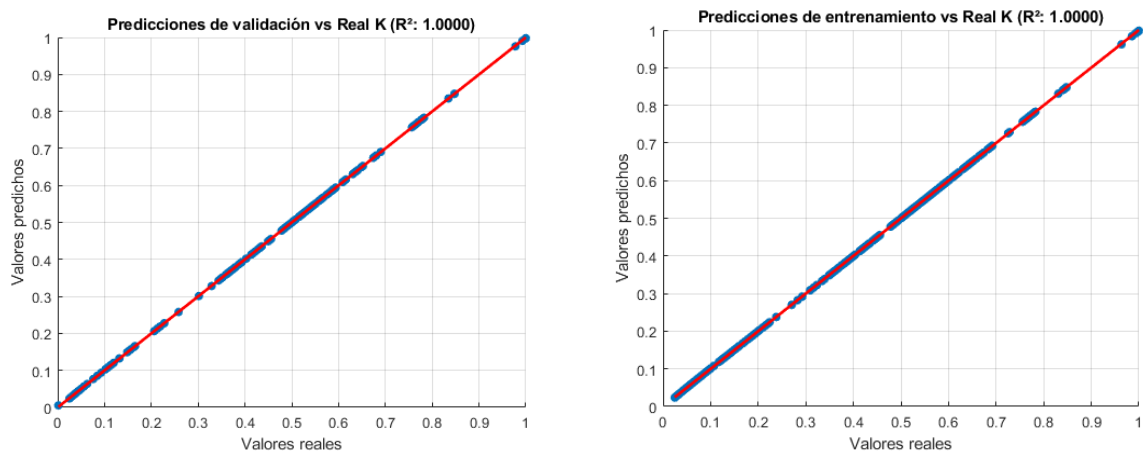


Figura 18: Gráficos de dispersión de la Red_k para la variable K, el grafico de la izquierda corresponde a los datos de validación y el de la derecha corresponde a los datos de entrenamiento.

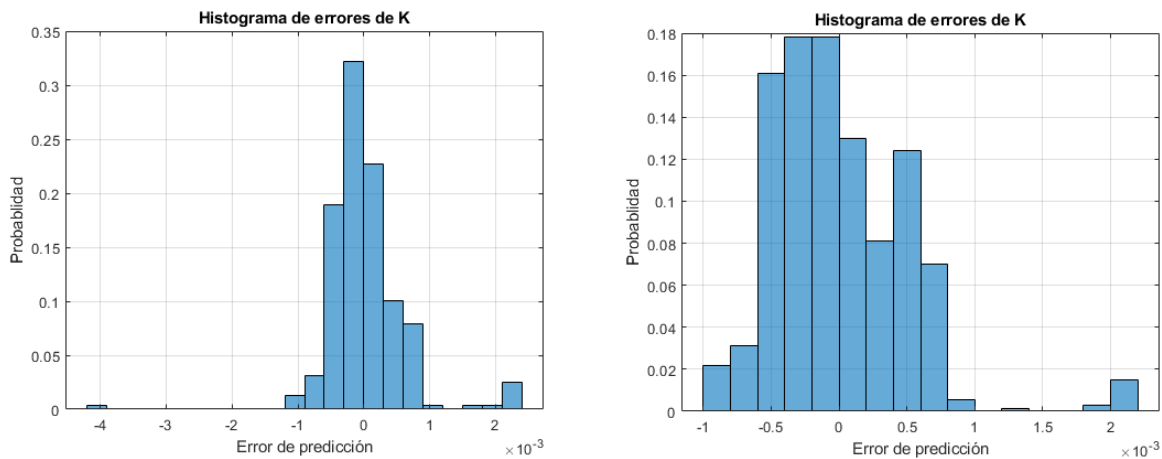


Figura 19: Gráficos de dispersión de la Red_k para la variable K, el grafico de la izquierda corresponde a los datos de validación y el de la derecha corresponde a los datos de entrenamiento.

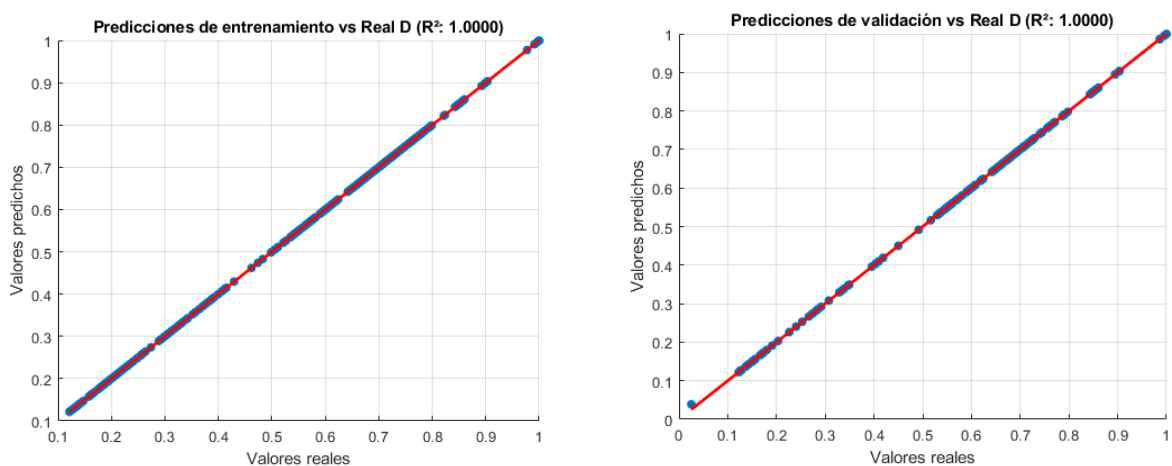


Figura 20: Gráficos de dispersión de la Red_d para la variable D, el grafico de la izquierda corresponde a los datos de entrenamiento y el de la derecha corresponde a los datos de validación.

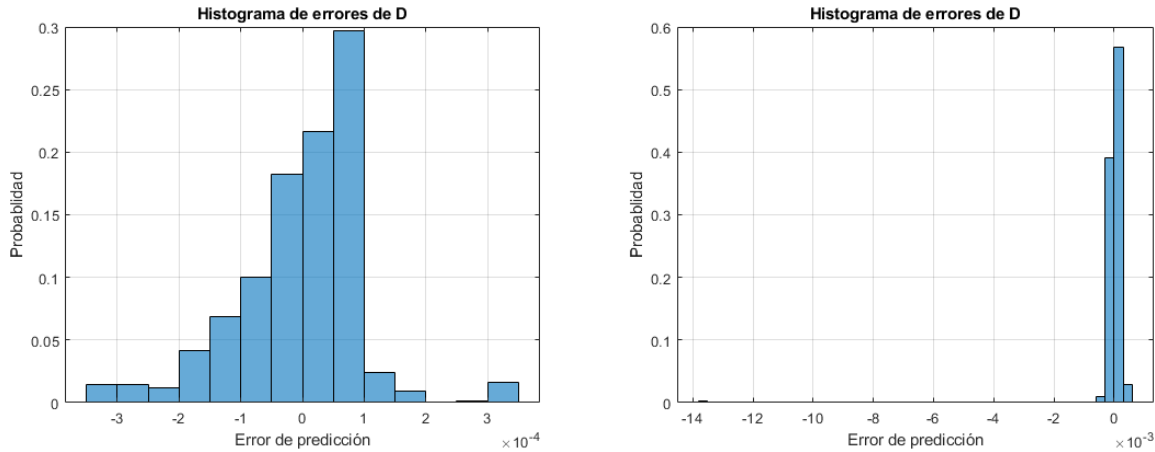


Figura 21: Gráficos de dispersión de la Red_d para la variable D, el gráfico de la izquierda corresponde a los datos de validación y el de la derecha corresponde a los datos de entrenamiento.

De la figura 17 se desprende que, en ambos casos, las curvas descienden y se estabilizan en valores bajos, lo que refleja un buen aprendizaje y una adecuada capacidad de generalización por parte de las redes Red_k y Red_d . Como consecuencia de lo anterior, en los gráficos de las figuras 18 y 20 se observa que los valores de R^2 obtenidos son iguales a 1, tanto para la red Red_k como para la Red_d , tanto en los datos de entrenamiento como en los de validación. Esto indica que las predicciones son perfectas en ambos casos, resultado que era esperable dada la buena capacidad de generalización de las redes y el relativamente pequeño volumen de datos.

Por otro lado, los histogramas de las figuras 19 y 21 muestran que la mayor parte de la distribución de los errores se concentra alrededor de 0, lo que sugiere que no existe sesgo en las predicciones. Sin embargo, se observan algunos valores que se alejan de este centro, lo que posiblemente se debe a que corresponden a los extremos del intervalo. Esto puede significar un peor ajuste en los extremos del intervalo con respecto a los valores de referencia. La causa de este mal ajuste en los extremos se debe a la distribución de los datos alimentados a las redes neuronales, presentados en la figura 22.

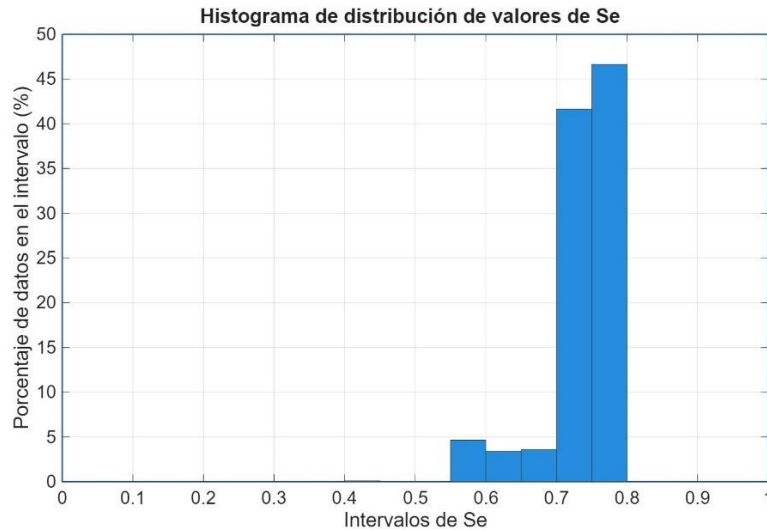


Figura 22: Histograma de distribución de valores de Se en el intervalo

Como se puede ver en la figura 22, la mayor cantidad de datos se encuentra en el intervalo de 0.7 a 0.8, constituyendo un 88.3% del total de los datos- Por el otro extremo del intervalo, los datos desde 0.4 a 0.5 solo constituyen un 0.9% del total de datos. Esto muestra la falta de datos en el intervalo de 0.4 a 0.5 en comparación al resto del intervalo, lo que condiciona la capacidad de predicción de las redes Red_k , Red_d y la Red_j .

A pesar de esto, en los histogramas de las figuras 19 y 21, los valores de porcentaje de error poseen simetría en 0, por lo que se evidencia un mejor ajuste. Adicionalmente, comparando los histogramas de validación y entrenamiento en las dos figuras, se observan que las distribuciones de los dos histogramas son similares si obviamos los valores de los extremos. Esto indica una buena capacidad de generalización. Sin embargo, las dos redes poseen alta varianza de los errores de predicción, lo que indica que posiblemente las predicciones sean inconsistentes en el tiempo, es decir, en caso de repetir la misma solicitud de predicción en dos instantes de tiempo diferentes.

5.2.2. Comparación entre modelos

A continuación, se presentan los perfiles de humedad predichos por cada modelo y las comparaciones de los modelos con redes incluidas con el modelo base.

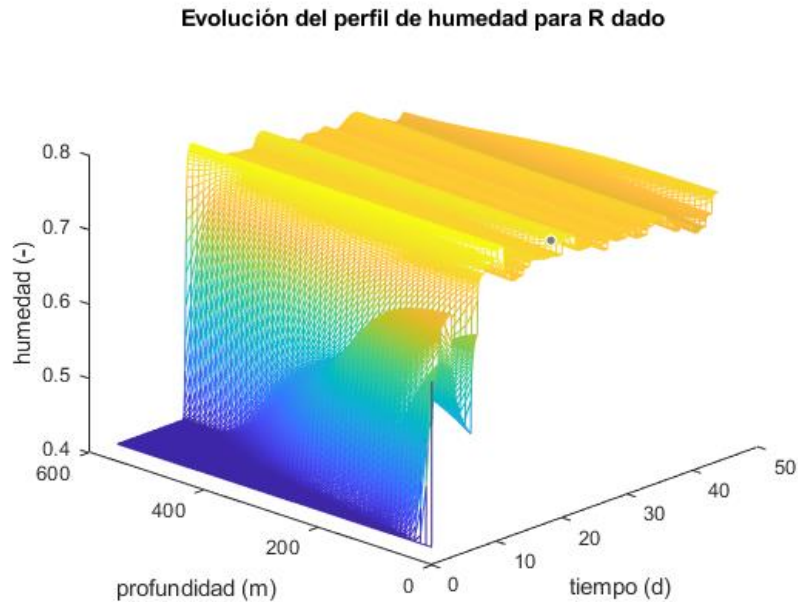


Gráfico 1: Perfil de humedad para R dado, obtenido con el modelo de Richards y curvas de retención de Van Genuchten.

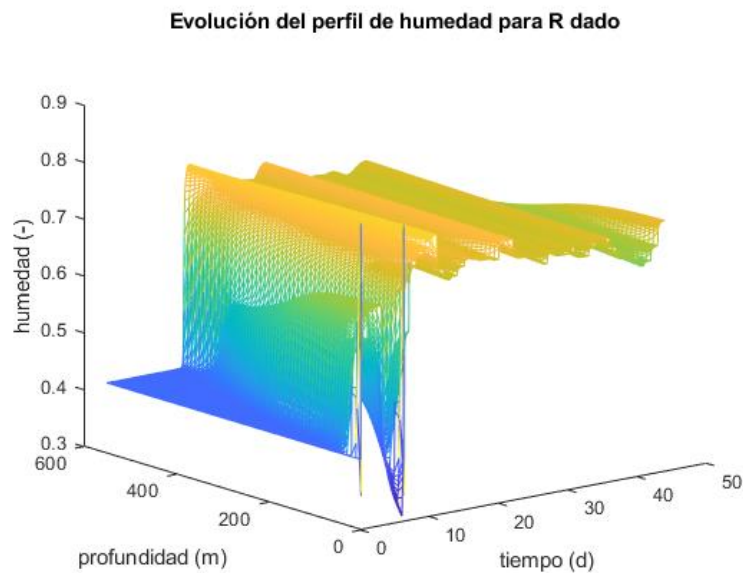


Gráfico 2: Perfil de humedad para R dado, obtenido con el modelo de Richards y red neuronal artificial *Red_J*.

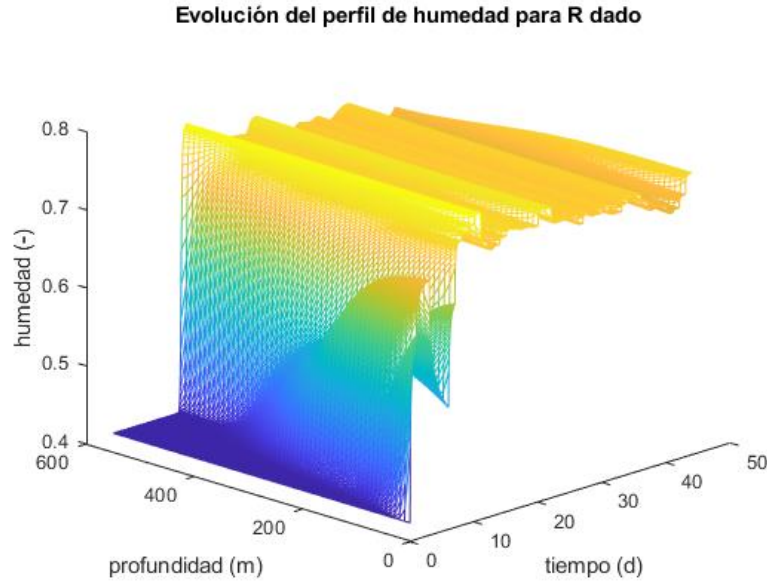


Gráfico 3 : Perfil de humedad para R dado, obtenido con el modelo de Richards y redes neuronales Red_k y Red_d .

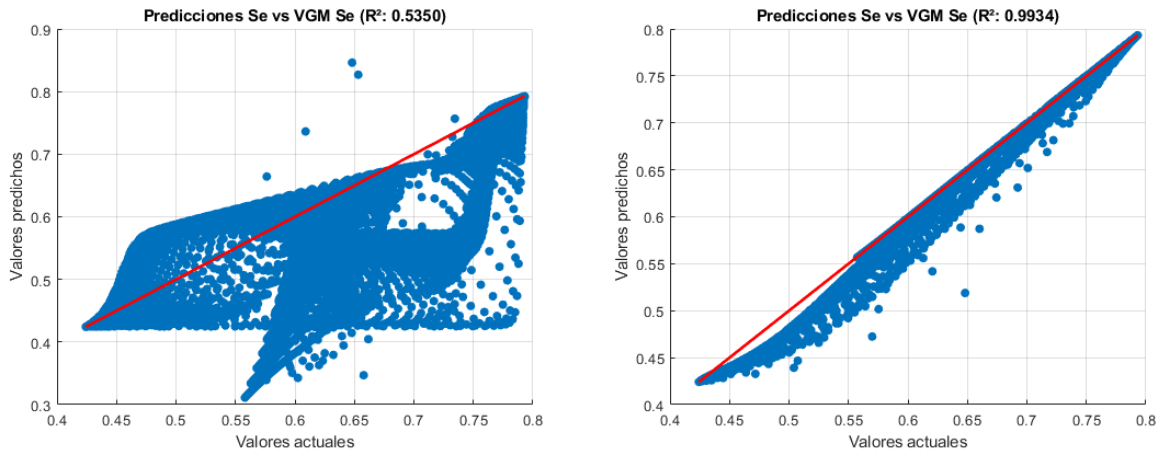


Figura 23: Gráficos de dispersión de la Red_j , Red_k y Red_d para la variable Se, el gráfico de la izquierda corresponde a la comparación entre el modelo base y el modelo usando la red Red_j , mientras que el de la derecha corresponde a la comparación entre el modelo base y el modelo usando las redes Red_k y Red_d .

En los gráficos obtenidos se observa que los gráficos 1 y 3 presentan comportamientos cualitativamente idénticos, mientras que el gráfico 2 muestra desviaciones, principalmente en la condición de borde superior ($z \approx 0$) entre los días 0 y 10, y para $t > 30$ días. Estas desviaciones se deben a las razones discutidas en el punto anterior.

Los gráficos de dispersión mostrados en la figura 22 fueron elaborados utilizando la totalidad de los perfiles de humedad. De estos, se desprende que el valor de R^2 obtenido para la comparación entre la red Red_j y el caso base es de 0.5350, lo que indica que las predicciones del modelo son aceptables, aunque con un margen de mejora considerable. En contraste, la figura de dispersión correspondiente al modelo que utiliza las redes Red_k y Red_d presenta un R^2 de 0.9934, lo que sugiere una predicción altamente precisa.

No obstante, en esta última figura se observa una cantidad de datos por debajo de la línea de idealidad, los cuales corresponden principalmente al conjunto de datos generados por las condiciones de borde. Estas condiciones son las que presentan mayor variabilidad durante la simulación. A pesar de esto, la diferencia entre estos datos y los datos obtenidos del modelo se considera aceptable.

5.3. Discusión sobre resultados

Los resultados obtenidos en el apartado anterior nos muestran que las redes neuronales Red_k y Red_d pueden predecir con precisión los valores de difusividad y conductividad hidráulica para la mayoría de los datos de humedad relativa, lo que resulta beneficioso para la simulación numérica del problema de Richards.

Una ventaja clave de estas redes neuronales sobre la curva de Van Genuchten radica en su diferenciabilidad: al emplear funciones de activación suaves como la tangente hiperbólica, las redes son continuamente derivables, a diferencia de la curva de Van Genuchten que presenta discontinuidades en su derivada.

Esta propiedad es crucial para aplicaciones de optimización y control predictivo, donde se busca determinar la trayectoria óptima de la tasa de riego que homogenice el perfil de humedad, que maximice la recuperación mineral y minimice la inestabilidad mecánica en esquemas de optimización que requieren modelos matemáticos diferenciables. Si bien la implementación de estos esquemas excede el alcance actual, representa una aplicación potencial de las redes desarrolladas. No obstante, es importante reconocer las limitaciones de la implementación realizada en este trabajo.

Por una parte, las redes con mejor valor de ajuste que son las redes Red_k y Red_d presentan una desviación en sus predicciones de las condiciones de borde del problema, como se evidencia en la figura 23. Como se dijo en el apartado 5.2.1, esto es debido a que en el entrenamiento se tienen menos datos en el intervalo de 0.4242 hasta 0.55 para realizar el ajuste, representando un 0.9% del total de los datos. Esto conlleva una menor capacidad de predicción alrededor de estos datos, generando errores en la predicción cercana a estos valores. Las motivaciones para la selección de ese conjunto de datos fueron dos.

La primera razón es que se escogió trabajar con esos datos ya que dentro de la simulación matemática expuesta por Cariaga et al (Cariaga, 2015), este conjunto de datos representa físicamente datos tomados de un sensor en una posición z dentro de la altura de la columna, a través del tiempo t dentro del total de tiempo simulado, en específico la posición 1 o al tope de la pila y

por un periodo de tiempo de 44 días. Entonces al escoger este conjunto de datos, se limita los datos utilizables para el entrenamiento de la red neuronal, ya que sería poco realista utilizar todos los datos disponibles de toda la simulación.

La segunda razón es que se seleccionaron estos datos debido a que presentan una variabilidad significativa durante todo el periodo de tiempo estudiado en la simulación. Esto es importante, ya que, para el entrenamiento, es conveniente realizarlo con la mayor cantidad de datos diferentes para poder predecir en un amplio rango de humedades.

Sin embargo y en retrospectiva, para obtener mejores ajustes para las redes neuronales se podría haber considerado los datos de otras particiones para así obtener un mejor ajuste en el rango de 0.4242 a 0.55 de humedad relativa.

Por otro lado, el entrenamiento de las redes se basó en datos simulados asumiendo una curva de retención específica, lo que introduce incertidumbre estructural inherente a los modelos empíricos. Por tanto, las redes obtenidas pueden heredar los sesgos de la curva de retención de referencia, comprometiendo su robustez y precisión predictiva.

Para abordar estas limitaciones, se plantean dos alternativas:

1. Enfoque experimental: Utilizar datos de laboratorio obtenidos mediante la ley de Darcy y la ecuación de Richards a partir de muestras reales de pilas de lixiviación. Esto eliminaría el sesgo del modelo empírico, aunque conlleva mayores costos económicos.
2. Enfoque híbrido: Implementar redes neuronales informadas por física (PINNs) que incorporen las ecuaciones de balance de masa durante el entrenamiento. Estas redes predecirían perfiles de humedad simulados que se compararían con datos experimentales o de curvas de retención, manteniendo consistencia física mientras reducen la dependencia de modelos empíricos.

Ambas estrategias buscan superar las limitaciones del enfoque actual, ya sea mediante una base experimental más sólida o mediante la incorporación de restricciones físicas en el proceso de aprendizaje automático. Cabe mencionar que, si bien la implementación de estas alternativas escapa del alcance del trabajo, se propone una posible metodología para los PINNs

Para el segundo enfoque, se necesita de un método de simulación diferente al de Crank-Nicolson, debido a que este es sensible a los valores iniciales de las variables a medir por lo que se proponen los siguientes modelos matemáticos y su comprobación.

5.3.1. Implementación del método matemático y problemática

Para implementar métodos de discretización dentro del algoritmo de optimización para entrenar a las redes neuronales, es necesario primero elegir el modelo más apropiado para la problemática considerando que, como se trata de un entrenamiento de red neuronal, el modelo debe ser robusto numéricamente y no tan sensible a desviaciones en los instantes iniciales.

Esto debido a que, en las primeras épocas de entrenamiento, la red neuronal arroja valores que solo están guiados por la inicialización aleatoria de las neuronas, por lo tanto, en una primera instancia, los valores de las variables a simular (D y K), son completamente aleatorios o pueden ser inferidos de otra fuente.

Esto se puede mitigar en cierta medida, mediante la normalización de los datos de salida, utilizando un valor que represente el orden de magnitud supuesto de la salida. Además, esto se puede mitigar restringiendo la arquitectura de la red neuronal, añadiendo una capa en la salida que actúe como filtro, para que entregue dentro del dominio deseado. Sin embargo, esto no es suficiente para solventar este problema, por lo que la robustez del modelo matemático tiene mayor importancia.

Debido a su complejidad matemática, y la sensibilidad al valor inicial, el modelo de Crank-Nicolson se consideró no apto para ser utilizado de esta manera, por lo que esto llevo al desarrollo del método de Euler explícito en el anexo D y Runge-Kutta de orden 4 en el anexo E aplicados al problema de Richards.

El método de Euler explícito es un método para la resolución numérica del modelo de Richards. Su principal atractivo es la facilidad de implementación ya que se hace una aproximación lineal de la solución basado en el valor anterior y la función conocida. Sin embargo, este método puede ser inestable si se tienen ecuaciones diferenciales de orden superior, como lo es el caso del modelo de Richards, por lo que puede ser necesario un método más preciso.

El método de Runge-Kutta de orden 4 es un método numérico que se aproxima a la solución del modelo mediante el cálculo en puntos intermedios para realizar un promedio ponderado de las pendientes y así obtener una mejor aproximación. Este método es más preciso, versátil y solo un poco más complejo de aplicar que el método de Euler explícito, sin embargo, es más demandante en el costo computacional debido al cálculo de múltiples puntos intermedios para realizar una sola predicción.

Para probar estos métodos y compararlos con la resolución y el perfil de humedad predicho por la combinación de Crank-Nicolson y el uso de las curvas de retención, expuesto por Cariaga et al (2015) se creó el siguiente algoritmo en MATLAB cuyo código se encuentra en el anexo L:

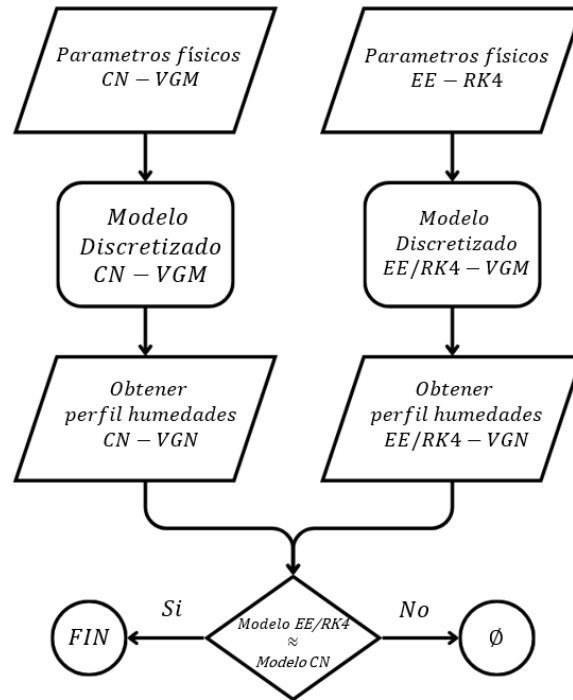


Figura 24: Algoritmo de comparación modelos matemáticos

Como se muestra en la figura 24, en este algoritmo se cargan los parámetros físicos y de discretización del anexo A al código, tanto para el modelo de Crank-Nicolson como para el modelo de Euler Explicito (EE) o el de Runge-Kutta de cuarto orden (RK4).

Luego se simula en paralelo para obtener el perfil de humedades predichos por cada modelo. Estos perfiles se analizan cualitativamente debido a que no es necesario obtener los valores precisos de este ejercicio, si no un comportamiento aproximadamente similar en ambos casos.

De ser comparables se concluiría que los modelos presentan comportamientos similares y se procedería a usar el modelo más numéricamente estable en el entrenamiento de las redes neuronales.

De no ser comparables tenemos un modelo insuficientemente robusto para la simulación matemática del problema de Richards, representado por un conjunto vacío ($\emptyset$), por lo que habría que evaluar la implementación de estos métodos u otro tipo de métodos numéricos para su resolución matemática.

5.3.2. Definición del algoritmo de entrenamiento

Para entrenar las redes neuronales informadas por procesos físicos se propone el siguiente algoritmo de entrenamiento y optimización visualizado en la figura 25, el código de dicho algoritmo fue desarrollado en Matlab y se encuentra en el anexo K:

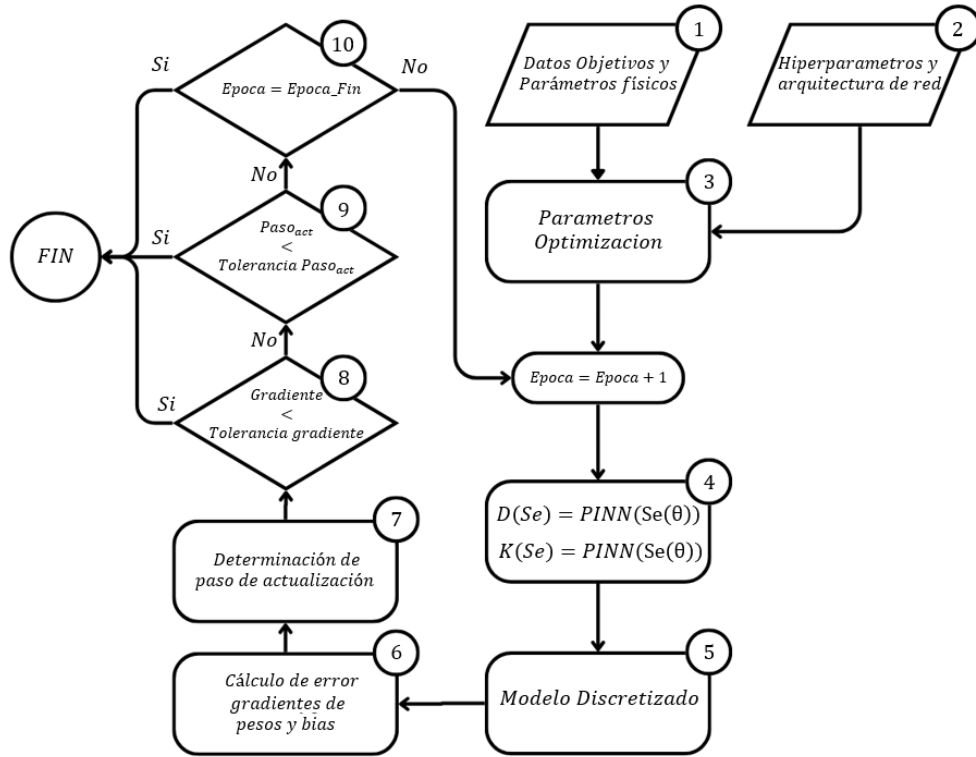


Figura 25: algoritmo de entrenamiento PINN (elaboración propia)

Al inicio del algoritmo, es necesario definir los datos objetivos Se y los parámetros físicos que describen el comportamiento de la columna (Paso 1).

Para ello, se utilizó el modelo matemático de Crank-Nicolson modificado, empleando la curva de retención de Van Genuchten para obtener los datos objetivos, junto con parámetros físicos de la columna extraídos de Cariaga et al. (2015), los cuales se detallan en el anexo (A).

Además, se establecen los hiperparámetros y la arquitectura de la red neuronal (Paso 2), basándose en los resultados obtenidos en la simulación preliminar descrita en la sección 5.1 de este informe.

Con estos elementos definidos, se configuran los parámetros de optimización, que incluyen la tolerancia mínima de la norma de gradientes, la tolerancia mínima del paso de actualización y el número máximo de épocas de optimización (Paso 3).

Dentro del ciclo de optimización, se evalúa la función de pérdida. Para ello, primero se simula la columna de lixiviación. La simulación comienza con la discretización de la ecuación utilizando N_t y N_z puntos discretos en el tiempo y el espacio, respectivamente. Luego, se utilizan las redes

neuronales sin entrenar para predecir los componentes D y K (Paso 4). Con estos componentes y el modelo discretizado, se simula un perfil de humedades (Paso 5) para todos los puntos discretos temporales.

A partir del perfil predicho, se calcula la norma L_2 de los residuos en los puntos de discretización, comparándolos con los datos objetivos Se. Con esta medida, se calcula el gradiente como la estimación de gradiente numérico unidimensional tomando el vector de los pesos y con espaciado entre los puntos igual al valor de pérdida obtenido (Paso 6). Posteriormente, el algoritmo de actualización L-BFGS predice el siguiente paso de actualización de los parámetros de la red neuronal utilizando los gradientes calculados (Paso 7).

Para que el algoritmo continúe actualizando, deben cumplirse tres condiciones:

1. El gradiente no debe ser menor que el parámetro de tolerancia de gradiente (Paso 8), evitando así la implosión de gradientes en las redes.
2. El paso de actualización no debe ser menor que el parámetro de tolerancia del paso de actualización (Paso 9), lo que previene cambios demasiado pequeños en la red.
3. La época actual debe ser menor que el número máximo de épocas definido (Paso 10), que marca el fin del proceso de optimización.

Este proceso se repite hasta que alguna de las condiciones de los pasos 8, 9 o 10 se cumpla, finalizando así la optimización.

Esta metodología para el entrenamiento de esta red neuronal guiada por las ecuaciones de Richards fue implementada como se muestra en el pseudo código en el anexo M. Sin embargo, no se llegó a una solución satisfactoria dentro del tiempo dispuesto para este trabajo de título, debido a la mala convergencia de los resultados de la red neuronal en comparación al método de Crank-Nicolson.

Esta mala convergencia es debido a que dentro del método de optimización L-BGFS, se calcula el paso optimo de actualización de los subparámetros (los pesos y bias) aproximando el inverso de la matriz Hessiana, donde se utiliza en conjunto con los gradientes de la red neuronal para calcular una dirección de actualización, en un enfoque cuasi-Newton.

Esto quiere decir que para obtener el siguiente paso de actualización se necesitan los gradientes de la red neuronal, que corresponden a la diferencia finita de los valores del error o residuos en los puntos de discretización, respecto de los valores de los pesos y bias de la red.

Dicho lo anterior, la razón de la mala convergencia está en el método de obtención de los gradientes, ya que, al introducir las ecuaciones de Richards como guía para el entrenamiento, se introduce un error intrínseco asociado a la integración de las ecuaciones dentro de la metodología.

Debido a que el objetivo de la función de pérdida es calcular un error entre los valores predichos y los datos de entrenamiento, esta diferencia también lleva asociada el error de integración y debido el gradiente se calcula como la diferencia entre este error calculado y la diferencia de los gradientes,

este error de integración se propaga dos veces, en cada subparámetros y en cada iteración del algoritmo.

Entonces esto podría explicar la mala convergencia obtenida, ya que, al propagarse el error dentro del gradiente, también lo hace para el paso de optimización en la siguiente iteración del algoritmo, resultando en una peor predicción en cada iteración hasta el punto de obtener valores físicamente incorrectos o nulos.

Como observación, también existe otro método de cálculo para los gradientes, que es la diferenciación automática. Este método se encuentra en Matlab dentro de la biblioteca de deeplearning toolbox, sin embargo, no fue posible implementarlo en el código dentro del tiempo de realización de este trabajo de título debido a la complejidad informática que implica la conciliación entre las herramientas de deeplearning y el método matemático.

6. Conclusiones y recomendaciones

Al integrar redes neuronales artificiales dentro de un modelo fenomenológico discretizado basado en la ecuación de Richards utilizando el método de Crank-Nicolson se cumplió el objetivo general del trabajo, mostrando que las redes neuronales son capaces de predecir la conductividad y la difusividad hidráulica, lo que permitió describir el perfil espaciotemporal de la humedad en una columna de lixiviación simulada.

Para lograr esto, primero se hizo un análisis de sensibilidad de los hiperparámetros de número de neuronas por capa y de número de capas de las redes neuronales en función de su capacidad de ajuste a los datos entregados. Este análisis se obtuvo que para obtener una red neuronal capaz de realizar un ajuste satisfactorio solo se necesita una capa y aproximadamente de 7 a 10 neuronas por capa para cada red neuronal.

De los resultados de este análisis se puede concluir que, aun dada la naturaleza no lineal de los datos entregados, la complejidad necesaria de las redes neuronales no es alta para obtener resultados satisfactorios, y si bien un mayor número de neuronas en general podría mejorar el ajuste, esto también conllevaría un aumento en el riesgo de sobreajuste de los datos.

Luego, se definió el uso de redes neuronales artificiales profundas (DNN) con arquitecturas específicas para predecir la conductividad y la difusividad hidráulica. Se estudió la diferencia entre dos tipos de arquitectura: una red con dos salidas, una para $K(\theta)$ y otra para $D(\theta)$, y dos redes separadas, cada una con una salida para $K(\theta)$ y $D(\theta)$ respectivamente.

Los resultados mostraron que las redes neuronales separadas con una salida para cada variable de conductividad hidráulica y difusividad tuvieron un desempeño significativamente mejor que la red con dos salidas.

Las redes separadas lograron un coeficiente de determinación (R^2) cercano a 1 tanto en los datos de entrenamiento como en los de validación, lo que indica una alta precisión en la predicción de las variables. Por otro lado, la red con dos salidas presentó un R^2 menor a 0.5, lo que sugiere un mal ajuste y una menor capacidad de generalización de datos.

Esto se atribuyó a la complejidad de predecir dos variables diferentes a partir de una misma entrada, lo que generó un sesgo sistemático en las predicciones. Por lo que se concluye que, para el problema presentado, es mejor entrenar dos redes por separado para la predicción de las variables de conductividad y difusividad hidráulica.

En cuanto a la implementación de las redes neuronales en el modelo de Crank-Nicolson, se observó que el modelo con las redes separadas fue capaz de reproducir el perfil de humedad. El R^2 obtenido en la comparación entre el modelo con curvas de retención y el modelo con redes neuronales fue de 0.9934, lo que indica una alta correlación entre las predicciones y los valores reales.

Por el contrario, el modelo con la red de dos salidas presentó desviaciones significativas en las condiciones de borde, especialmente en los primeros días de simulación y en tiempos prolongados.

De esto se concluye que las ANN demostraron ser una alternativa viable a los modelos basados en curvas de retención empíricas, que presentan desafíos numéricos debido a su no diferenciabilidad y complejidad computacional. Las redes neuronales, al ser diferenciables, facilitan su integración en esquemas de optimización y control predictivo, lo que es crucial para aplicaciones industriales.

Sin embargo, aunque las ANN entrenadas con datos simulados mostraron alta precisión, heredaron la incertidumbre estructural de las curvas de retención utilizadas para generarlos. Esto subraya la importancia de validar estos modelos con datos experimentales reales para garantizar su robustez en condiciones operativas reales.

Adicionalmente, las redes entrenadas presentaron en todos los casos una deficiencia para generalizar datos en el intervalo de humedades de 0.4242 hasta 0.55 aproximadamente. Posiblemente esto es debido a que durante el entrenamiento los datos utilizados eran menor en cantidad respecto a los datos en valores de humedad mayor a los mencionados. Esto indica la importancia del análisis de los datos usados para obtener mejores resultados.

Como recomendaciones se indica explorar el uso de redes neuronales informadas por procesos físicos (PINN) en futuras investigaciones. También, se sugiere la nueva formulación del método de entrenamiento de redes neuronales guiadas por ecuaciones físicas utilizando esta vez la herramienta de diferenciación automática para el cálculo de gradientes. Además, se sugiere la recolección de datos experimentales reales para entrenar y validar las redes neuronales, lo que permitiría una mejor generalización, robustez y aplicabilidad en condiciones reales de operación.

7. Referencias

- Araya, B. (2024). *DESARROLLO DE UN SISTEMA DE CONTROL DE HUMEDAD EN UNA COLUMNA DE LIXIVIACIÓN A ESCALA LABORATORIO*.
- Bear, J. (1972). *Dynamics of fluids in porous media*. New York, American Elsevier Pub. Co.
- Benjamín, O., Araya, B., Quintanilla, P., & Navia, D. (2025). Saturation Regulation in Heap Leaching: A nonlinear model Predictive control approach. SSRN. <https://doi.org/10.1016/j.mineng.2025.109346>
- Cariaga, E., Martínez, R., & Sepúlveda, M. (2015). Estimation of hydraulic parameters under unsaturated flow conditions in heap leaching. *Mathematics and Computers in Simulation*, 109, 20–31. <https://doi.org/10.1016/j.matcom.2014.07.006>
- COCHILCO. (2023). Proyección de la producción esperada de cobre 2023-2034. In *www.cochilco.cl* (pp. 1–10). <https://www.cochilco.cl/Mercado%20de%20Metales/Proyecci>
- Cochilco. (2024, May 15). *Boletín Mensual Electrónico | Cochilco*. Wwww.cochilco.cl. <https://www.cochilco.cl/Paginas/Estadisticas/Publicaciones/BoletinMensualElectronico.aspx>
- Codelco. (2018). *LIXIVIACIÓN “El riego” LIXIVIACIÓN “El riego.”* https://www.codelcoeduca.cl/codelcoeduca/site/artic/20190109/asocfile/20190109005316/lixiviacion_media_t_cnico_080119.pdf
- D. Wulfsohn, Adams, B. A., & Fredlund, D. G. (1996). Application of unsaturated soil mechanics for agricultural conditions. *Canadian Agricultural Engineering*, 38(3), 173–181.
- Daud, O., Correa, M., Estay, H., & Ruízdelsolar, J. (2021). Monitoring and Controlling Saturation Zones in Heap Leach Piles Using Thermal Analysis. *Minerals*, 11(2). <https://doi.org/10.3390/min11020115>
- Dixon, D. (2003). Heap leach modeling The current state of the art. *Proceedings of the TMS Fall Extraction and Processing Conference*, 1, 289–314.
- Domic Mihovilovic, E. (2001). *Hidrometalurgia : fundamentos, procesos y aplicaciones*.

- Emilio Cariaga, Rubén Martínez, and Mauricio Sepúlveda. *Estimation of hydraulic parameters under unsaturated flow conditions in heap leaching. Mathematics and computers in simulation*, 109:20–31, 2015.
- Flores, V., Keith, B., & Leiva, C. (2020). Using Artificial Intelligence Techniques to Improve the Prediction of Copper Recovery by Leaching. *Journal of Sensors*, 2020, 2454875. <https://doi.org/10.1155/2020/2454875>
- Fredlund, D. G., Harianto Rahardjo, Fredlund, M. D., & Wiley, J. (2012). *Unsaturated soil mechanics in engineering practice*. John Wiley & Sons.
- GARDNER, W. R. (1958). SOME STEADY-STATE SOLUTIONS OF THE UNSATURATED MOISTURE FLOW EQUATION WITH APPLICATION TO EVAPORATION FROM A WATER TABLE. *Soil Science*, 85(4), 228–232. <https://doi.org/10.1097/00010694-195804000-00006>
- Géron, A. (2019). *Hands-on machine learning with Scikit-Learn and TensorFlow concepts, tools, and techniques to build intelligent systems* (2nd ed.). O'Reilly Media, Inc.
- Henríquez, Marcelo. (2021). Diseño de un Sistema de Control Predictivo basado en Modelo Fenomenológico para Controlar la Humedad en Pilas de Lixiviación.
- Ivan Depina, Saket Jain, Sigurdur Mar Valsson & Hrvoje Gotovac (2022) *Application of physics-informed neural networks to inverse problems in unsaturated groundwater flow*, Georisk: Assessment and Management of Risk for Engineered Systems and Geohazards, 16:1, 21-36, DOI: 10.1080/17499518.2021.1971251
- McBride, D., Gebhardt, J. E., Croft, T. N., & Cross, M. (2016). Modeling the hydrodynamics of heap leaching in subzero temperatures. *Computational Modelling*, 90, 77–88. <https://doi.org/10.1016/j.mineng.2015.11.005>
- McBride, D., Gebhardt, J. E., & Cross, M. (2012). A comprehensive gold oxide heap leach model: Development and validation. *Hydrometallurgy*, 113/114, 98–108. <https://doi.org/10.1016/j.hydromet.2011.12.003>
- M. Davis, Numerical Methods and Modeling for Chemical Engineers, second ed., John Wiley & Son, USA, (1990).
- Revista Minería Chilena. (2023, October 17). *Revista Minería Chilena 508 by B2B MEDIA GROUP - Issuu*. Issuu.com. https://issuu.com/csa2020/docs/mch_508/8

- Robertson, S. W., Staden, van, Cherkaev, A., & Petersen, J. (2022). Properties governing the flow of solution through crushed ore for heap leaching. *Hydrometallurgy*, 208, 105811. <https://doi.org/10.1016/j.hydromet.2021.105811>
- RuizdelSolar, J., Daud, O., Correa, M., & Torres, M. (2013). *Controlling the irrigation flow in heap leach piles of El Soldado mine by canopy system with thermal camera*.
- Sánchez, J. (2024, May 13). *Hidrología*. Hidrología. <https://hidrologia.usal.es/index.html>
- Sun, T., Chen, F., Zhong, L., Liu, W., & Wang, Y. (2019). GISbased mineral prospectivity mapping using machine learning methods: A case study from Tongling ore district, eastern China. *Ore Geology Reviews*, 109, 26–49. <https://doi.org/10.1016/j.oregeorev.2019.04.003>
- T, T. F. (2006). Clean two- and three-dimensional analytical solutions of Richards' equation for testing numerical solvers. *Water Resour. Res.*, 42(8). <https://doi.org/10.1029/2005WR004638>
- Wang, L., Yin, S., & Deng, B. (2021). Understanding the Effect of Stepwise Irrigation on Liquid Holdup and Hysteresis Behavior of Unsaturated Ore Heap. *Minerals*, 11(11). <https://doi.org/10.3390/min11111180>

8. Anexos

8.1. Anexo A, parámetros físicos utilizados y datos utilizados

Tabla 1: Parámetros físicos y discretos (Cariaga, 2015)

Parámetro	símbolo	Valor	Unidad
Tiempo total	T	44	día
Altura	H	550	cm
Humedad inicial	$\theta_{inicial}$	0.14	cm ³ /cm ³
Humedad de saturación	θ_s	0.33	cm ³ /cm ³
Humedad residual	θ_R	0	cm ³ /cm ³
Conductividad hidráulica de saturación	K_s	170	cm ³ /día
Parámetro Van Genuchten	α	0.035	1/cm
Parámetro Van Genuchten	m	2.267	-
Tamaño paso temporal	Δt	1/24	día
Tamaño paso espacial	Δz	10	cm

Tabla 2: Caudales de riego para el tiempo de simulación (Cariaga, 2015)

Día	Caudal, m ³ /día	Día	Caudal, m ³ /día	Día	Caudal, m ³ /día	Día	Caudal, m ³ /día
1	12	12	27	23	26	34	25
2	19	13	23	24	26	35	27
3	19	14	24	25	26	36	27
4	0	15	27	26	25	37	23
5	0	16	30	27	22	38	19
6	0	17	29	28	26	39	20
7	12	18	28	29	24	40	19
8	32.5	19	28	30	24	41	16
9	36	20	27	31	26	42	17
10	39	21	26	32	22	43	21
11	46.5	22	25	33	22	44	30

8.2. Anexo B, Resultados sensibilización de hiperpárametros

Tabla 3: Resultados de sensibilización de hiperpárametros para K. (Fuente: Matlab)

Numero de capas	Numero de neuronas por capa	Error normalizado
1	1	0.001642365
1	2	0.002663257
1	3	0.000354692
1	4	0.001748705
1	5	0.004164546
1	6	0.002250622
1	7	0.000513231

Tabla 4: Resultados de sensibilización de hiperpárametros para D. (Fuente: Matlab)

Numero de capas	Numero de neuronas por capa	Error normalizado
1	1	0.00154971
1	2	0.001631718
1	3	0.001945695
1	4	0.002231348
1	5	0.001530464
1	6	0.00103313
1	7	0.001279548
1	8	0.001579459
1	9	0.002554837
1	10	0.000588449

8.3. Anexo C, Crank-Nicolson Modificado

Este método de diferencias finitas presentado por (Cariaga, 2015) para la ecuación de Richards es un método numérico de diferencias finitas basado en el método de Crank-Nicolson donde se aproximan las soluciones para θ con una etapa correctora y predictora. Para la discretización de la ecuación (3.8a) se tiene un dominio espacial $[0, H]$ y un dominio temporal $[0, T]$. Sean N_z y N_t el número de subintervalos para $[0, H]$, $[0, T]$ respectivamente, se define $\Delta z = H/N_z$ y $\Delta t = T/N_t$ y θ_i^j es una aproximación de $\theta(i\Delta z, j\Delta t)$ con $i \in [0, \dots, N_z]$ y $j \in [0, \dots, N_t]$. Entonces la discretización de la ecuación (3.8a) en los intervalos anteriores está dada por (Cariaga, 2015):

$$\frac{(\theta_i^{j+1} - \theta_i^j)}{\Delta t} = \frac{1}{2}(E_i^{j+1} + E_i^j) - \frac{1}{2}(F_i^{j+1} + F_i^j) \quad (5.5)$$

Donde:

$$E_i^{j+1} = \frac{(D_{ff} * (\theta_{i+1}^{j+1} - \theta_i^{j+1}) - D_{bf} * (\theta_i^{j+1} - \theta_{i-1}^{j+1}))}{(\Delta z)^2}$$

$$E_i^j = \frac{(D_f * (\theta_{i+1}^j - \theta_i^j) - D_b * (\theta_i^j - \theta_{i-1}^j))}{(\Delta z)^2}$$

$$F_i^m = \frac{(K_{i+1}^m - K_{i-1}^m)}{(2\Delta z)}, \quad m = j, j+1$$

Con $D_{ff} = \frac{(D_{i+1}^{j+1} + D_i^{j+1})}{2}$, $D_{bf} = \frac{(D_i^{j+1} + D_{i-1}^{j+1})}{2}$, $D_f = \frac{(D_{i+1}^j + D_i^j)}{2}$, $D_b = \frac{(D_i^j + D_{i-1}^j)}{2}$, $D_i^j = D(\theta_i^j)$, $K_i^j = K(\theta_i^j)$. Con las condiciones de borde (3.8c), (3.8d) para $i = 0$ e $i = N_z$, están dadas por:

$$-D_0^j \frac{(\theta_0^j - \theta_{-1}^j)}{\Delta z} + K_0^j = R^j, \quad \forall j = 0, \dots, N_t \quad (5.6)$$

$$\frac{(\theta_{N_z+1}^j - \theta_{N_z}^j)}{\Delta z} = 0, \quad \forall j = 0, \dots, N_t \quad (5.7)$$

Donde θ_{-1}^j y $\theta_{N_z+1}^j$ son incógnitas auxiliares. Este método es de segundo orden en el tiempo. Por otro lado, la discretización en el espacio es de primer orden y la difusión de coeficientes no es constante. Por esta razón la discretización de primer orden es suficiente para las ecuaciones (5.6) y (5.7)

$$A(\theta^{j+1}) \theta^{j+1} = b(\theta^j) + c(\theta^{j+1}) \quad (5.8)$$

Donde $\theta^{j+1} = (\theta_i^{j+1})_{(N_z+1) \times 1}$ y las entradas $A(\theta^{j+1})$, $b(\theta^j)$ y $c(\theta^{j+1})$ están definidas a continuación:

$$A = \begin{bmatrix} 1 + w_1(D_{ff} + D_{bf}) & -w_1 D_{ff} & 0 & \dots & \dots \\ -w_1 D_{bf} & 1 + w_1(D_{ff} + D_{bf}) & -w_1 D_{ff} & 0 & \dots \\ \dots & \dots & \dots & \dots & \dots \\ \dots & 0 & -w_1 D_{bf} & 1 + w_1(D_{ff} + D_{bf}) & -w_1 D_{ff} \\ \dots & \dots & 0 & -w_1 D_{bf} & 1 + w_1(D_{ff} + D_{bf}) \end{bmatrix} \quad (5.9)$$

Donde $w_1 = \frac{\Delta t}{(2(\Delta z)^2)}$. Las entradas de $b(\theta^j)_{(N_z+1) \times 1}$ para $i \in [0, \dots, N_z]$ están dadas por

$$b_{i+1} = \theta_i^j + w_1 [D_f(\theta_{i+1}^j - \theta_i^j) - D_b(\theta_i^j - \theta_{i-1}^j)] - w_2(K_{i+1}^j - K_{i-1}^j), \quad (5.10)$$

Donde $w_2 = \frac{\Delta t}{4\Delta z}$ y las entradas de $c(\theta^{j+1})_{(N_z+1) \times 1}$ para $i \in [0, \dots, N_z]$ son

$$c_{i+1} = -w_2(K_{i+1}^{j+1} - K_{i-1}^{j+1}). \quad (5.11)$$

Notar que para $i = 0$

$$c_1 = -w_2(K_1^{j+1} - K_{-1}^{j+1}) + w_1 D_{bf} \theta_{-1}^{j+1} \quad (5.12)$$

Donde θ_{-1}^{j+1} se calcula de la condición de borde (5.6), y para $i = N_z$

$$c_{N_z+1} = -w_2 (K_{N_z+1}^{j+1} - K_{N_z-1}^{j+1}) + w^1 D_{ff} \theta_{N_z+1}^{j+1} \quad (5.13)$$

Donde $\theta_{N_z+1}^{j+1}$ se calcula de la condición de borde (5.7). Este sistema de ecuaciones es no lineal, para poder resolverlo se utiliza el método corrector predictor, el cual consiste en aproximar el sistema a una forma lineal mediante iteraciones sucesivas. Este enfoque permite estimar una solución inicial para luego refinarla hasta que se satisfaga un criterio de convergencia. El valor auxiliar obtenido para cada iteración se denota por θ_{aux} y se utiliza un estimador intermedio para calcular θ^{j+1} en el siguiente paso. El método comienza calculando:

$$A(\theta^j)\theta_{aux} = b(\theta^j) + c(\theta^j); \quad (5.14)$$

Para calcular θ_{aux} , luego θ^{j+1} es calculado con:

$$A(\theta^j)\theta^{j+1} = b(\theta^j) + c(\theta_{aux}); \quad (5.15)$$

Este proceso se repite hasta que se cumpla el siguiente criterio de convergencia:

$$\frac{\theta^{j+1} - \theta_{aux}}{\theta^{j+1}} \leq \varepsilon; \quad (5.16)$$

Donde ε es un valor previamente definido para establecer el nivel de precisión del sistema.

8.4. Anexo D, Método de Euler explícito

Para su discretización se tiene que su dominio espacial es $[0,1]_\xi$ y su dominio temporal es $[0,1]_\tau$ para ξ y τ respectivamente. Sean N_ξ y N_τ el número de subintervalos para $[0,1]_\xi$, $[0,1]_\tau$ respectivamente, se define para la discretización espacial $\Delta\xi = \frac{z}{H}$ con $z \in [0, H]$, $\Delta\tau^k = \frac{t}{\Delta t}$ con $t \in [(k-1)\Delta t, k\Delta t]$, $k = [1, \dots, T]$ y Se_i^j que es una aproximación de $Se(i\Delta\xi, j\Delta\tau^k)$ con $i \in [0, \dots, N_\xi]$ y $j \in [0, \dots, N_\tau]$. Se considera la ecuación (5.20) para describir la evolución de la humedad Se . Para aproximar sus elementos diferenciales se utilizan diferencias finitas centradas debido a la presencia de la segunda derivada espacial en el término Se , descritas a continuación:

$$\frac{\partial Se}{\partial \tau} \approx \frac{Se_i^{j+1} - Se_i^j}{\Delta\tau} \quad (5.24)$$

$$\frac{\partial D}{\partial \xi} \approx \frac{D_{i+1}^j - D_{i-1}^j}{2\Delta\xi} \quad (5.25)$$

$$\frac{\partial Se}{\partial \xi} \approx \frac{Se_{i+1}^j - Se_{i-1}^j}{2\Delta\xi} \quad (5.26)$$

$$\frac{\partial K}{\partial \xi} \approx \frac{K_{i+1}^j - K_{i-1}^j}{2\Delta\xi} \quad (5.27)$$

$$\frac{\partial^2 Se}{\partial \xi^2} \approx \frac{Se_{i+1}^j - 2Se_i^j + Se_{i-1}^j}{\Delta \xi^2} \quad (5.28)$$

Sustituyendo estas aproximaciones en la ecuación (5.20) se obtiene:

$$\begin{aligned} \frac{H^2}{\Delta t} \frac{(Se_i^{j+1} - Se_i^j)}{\Delta \tau} &= \frac{(D_{i+1}^j - D_{i-1}^j)(Se_{i+1}^j - Se_{i-1}^j)}{(2\Delta \xi)^2} \\ &+ \frac{D_i^j (Se_{i+1}^j - 2Se_i^j + Se_{i-1}^j)}{\Delta \xi^2} - \left(\frac{H}{\Delta \theta} \right) \frac{(K_{i+1}^j - K_{i-1}^j)}{2\Delta \xi} \end{aligned} \quad (5.29)$$

Despejando para la variable Se_i^{j+1} se obtiene:

$$Se_i^{j+1} = Se_i^j + \left(\frac{\Delta t \Delta \tau}{H^2} \right) * \left[\frac{(D_{i+1}^j - D_{i-1}^j)(Se_{i+1}^j - Se_{i-1}^j)}{(2\Delta \xi)^2} + D_i^j \frac{(Se_{i+1}^j - 2Se_i^j + Se_{i-1}^j)}{\Delta \xi^2} - \left(\frac{H}{\Delta \theta} \right) \frac{K_{i+1}^j - K_{i-1}^j}{2\Delta \xi} \right] \quad (5.30)$$

Esta expresión permite calcular el valor de Se para $j + 1$ en función de los valores de la iteración actual j . Adicionalmente para las condiciones de borde (5.22) aplicando las aproximaciones centrales se obtiene lo siguiente:

$$\left(-D_i^j \frac{\Delta \theta}{H} \right) \left(\frac{Se_2^j - Se_0^j}{2\Delta \xi} \right) + K_i^j = R^k ; \forall i = 1 \wedge j > 0, k \in [t_0, t_f] \quad (5.31)$$

Despejando para Se_0^j se obtiene:

$$Se_0^j = Se_2^j + \frac{2\Delta \xi (R^k - K_i^j) H}{(D_i^j \Delta \theta)} ; \forall i = 1 \wedge j > 0, k \in [t_0, t_f] \quad (5.32)$$

Para la condición de borde (5.23) se tiene:

$$\frac{Se_2^j - Se_0^j}{2\Delta \xi} = 0 ; i = N_\xi \wedge j > 0 \quad (5.33)$$

8.5. Anexo E, Método de Runge-Kutta

La discretización del problema de Richards para este método es idéntica a la desarrollada en el punto anterior. El método Runge-Kutta (RK) desarrollado en este apartado corresponde al método de cuarto orden. Para un problema de valor inicial se define una función $f(x, y)$ tal que $y'(t) = f(x, y)$, $y(x_0) = x_0$. Entonces el método RK4 está dado por la siguiente ecuación:

$$y_{i+1} = y_i + \frac{1}{6} h (k_1 + 2k_2 + 2k_3 + k_4) \quad (5.34)$$

Donde

$$k_1 = f(x_i, y_i) \quad (5.35)$$

$$k_2 = f(x_i + \frac{1}{2}h, y_i + \frac{1}{2}k_1h) \quad (5.36)$$

$$k_3 = f(x_i + \frac{1}{2}h, y_i + \frac{1}{2}k_2h) \quad (5.37)$$

$$k_4 = f(x_i + h, y_i + k_3h) \quad (5.38)$$

Donde h es el tamaño del intervalo. Entonces utilizando diferencias finitas centradas idénticamente al modelo del punto anterior junto con las ecuaciones (5.20), (5.22) y (5.23), se obtiene el siguiente modelo:

$$Se_{i+1} = Se_i + \frac{1}{6}\Delta\tau^k(k_1 + 2k_2 + 2k_3 + k_4) \quad (5.39)$$

Donde

$$k_{1,i} = \left(\frac{\Delta t}{H^2}\right) * \left[\frac{(D_{i+1}^j - D_{i-1}^j)(Se_{i+1}^j - Se_{i-1}^j)}{(2\Delta\xi)^2} + D_i^j \frac{(Se_{i+1}^j - 2Se_i^j + Se_{i-1}^j)}{\Delta\xi^2} - \left(\frac{H}{\Delta\theta}\right) \frac{K_{i+1}^j - K_{i-1}^j}{2\Delta\xi} \right] \quad (5.40)$$

$$k_{2,i} = \left(\frac{\Delta t}{H^2}\right) * \left[\frac{(\dot{D}_{i+1,1}^j - \dot{D}_{i-1,1}^j)(\dot{Se}_{i+1,1}^j - \dot{Se}_{i-1,1}^j)}{(2\Delta\xi)^2} + \dot{D}_{i,1}^j \frac{(\dot{Se}_{i+1,1}^j - 2\dot{Se}_{i,1}^j + \dot{Se}_{i-1,1}^j)}{\Delta\xi^2} - \left(\frac{H}{\Delta\theta}\right) \frac{\dot{K}_{i+1,1}^j - \dot{K}_{i-1,1}^j}{2\Delta\xi} \right] \quad (5.41)$$

$$k_{4,i} = \left(\frac{\Delta t}{H^2}\right) * \left[\frac{(\dot{D}_{i+1,2}^j - \dot{D}_{i-1,2}^j)(\dot{Se}_{i+1,2}^j - \dot{Se}_{i-1,2}^j)}{(2\Delta\xi)^2} + \dot{D}_{i,2}^j \frac{(\dot{Se}_{i+1,2}^j - 2\dot{Se}_{i,2}^j + \dot{Se}_{i-1,2}^j)}{\Delta\xi^2} - \left(\frac{H}{\Delta\theta}\right) \frac{\dot{K}_{i+1,2}^j - \dot{K}_{i-1,2}^j}{2\Delta\xi} \right] \quad (5.42)$$

$$k_{4,i} = \left(\frac{\Delta t}{H^2}\right) * \left[\frac{(\bar{D}_{i+1}^j - \bar{D}_{i-1}^j)(\bar{Se}_{i+1}^j - \bar{Se}_{i-1}^j)}{(2\Delta\xi)^2} + \bar{D}_i^j \frac{(\bar{Se}_{i+1}^j - 2\bar{Se}_i^j + \bar{Se}_{i-1}^j)}{\Delta\xi^2} - \left(\frac{H}{\Delta\theta}\right) \frac{\bar{K}_{i+1}^j - \bar{K}_{i-1}^j}{2\Delta\xi} \right] \quad (5.43)$$

Donde para k_2 y k_3 los términos de tipo $\dot{U}_{i,n}^j$ están representados por $\dot{U}_{i,n}^j = U_i^j + \frac{\Delta\tau^k}{2}k_{n,i}$, de igual manera para k_4 los términos de tipo $\bar{U}_i^j$ están representados por $\bar{U}_i^j = U_i^j + \Delta\tau^k k_{3,i}$, en ambos casos U representando las variables D, K y Se, difusividad hidráulica, conductividad hidráulica y humedad relativa adimensional respectivamente. Las condiciones de borde (5.22) y (5.23), son idénticas a las obtenidas en las ecuaciones (5.32) y (5.33).

8.6. Anexo F, Reformulación de la integración de las redes al modelo fenomenológico

La implementación de una red neuronal informada por procesos físicos o PINN comienza por la construcción de una red neuronal tal que $\hat{u}(x; \chi)$ sea una aproximación de $u(x)$ donde t y z son considerados componentes dentro de x , con T conteniendo el dominio temporal. La entrada x y el vector de salida tienen las mismas dimensiones que u , y los pesos y bias de la red están denotadas por $\chi = W, b$. Siguiendo las especificaciones de la red, entonces también es necesario asegurar que la red satisface las condiciones de borde e iniciales de la PDE. Esto es implementado discretizando el espacio y restringiendo las soluciones $\hat{u}(x; \chi)$ a el set de puntos de colocación $\mathcal{H}$ dentro del dominio de T . Teniendo en cuenta lo anterior, la diferencia entre la red neuronal $\hat{u}(x; \chi)$ y las restricciones físicas se mide con la función de pérdida, definida como la suma ponderada de la norma de L_2 residuales en los puntos de colocación en el dominio y en las condiciones de borde (Depina et al, 2022)

$$\mathcal{L}(\chi; \mathcal{H}) = w_d \mathcal{L}_d(\chi; \mathcal{H}_d) + w_{cb} \mathcal{L}_{cb}(\chi; \mathcal{H}_{cb}) \quad (5.2a)$$

Donde w_d y w_{cb} son los pesos, $\mathcal{H}_d$ y $\mathcal{H}_{cb}$ son los puntos de colocación en el dominio y la condición de borde y:

$$\mathcal{L}_d(\chi; \mathcal{H}_d) = \frac{1}{|\mathcal{H}_d|} \sum_{x \in \mathcal{H}_d} \left\| f \left(x, \lambda, \frac{\partial \hat{u}}{\partial x}, \dots, \frac{\partial \hat{u}}{\partial x_d}, \frac{\partial^2 \hat{u}}{\partial x_1 \partial x_1} \dots \frac{\partial^2 \hat{u}}{\partial x_d \partial x_d} \right) \right\|_2^2 \quad (5.2b)$$

$$\mathcal{L}_{cb}(\chi; \mathcal{H}_{cb}) = \frac{1}{|\mathcal{H}_{cb}|} \sum_{x \in \mathcal{H}_{cb}} \|\mathcal{B}(\hat{u}, x)\|_2^2 \quad (5.2c)$$

Sin embargo, para nuestro caso las redes neuronales a generar consisten en $\hat{D}(\theta; \chi_D)$ y $\hat{K}(\theta; \chi_K)$. Esto genera dentro del problema una recursividad, ya que para obtener la simulación de los valores de D y K se debe contar con la simulación de las humedades relativas Se , pero para obtener la simulación de Se , debemos contar con D y K . Entonces la relación de K y D será indirecta tal que $\widehat{Se}(\theta; \chi_D, \chi_K)$, lo que significaría que dada una predicción aproximada de D y K se obtenga una predicción de Se . Por lo tanto, la función de pérdida a utilizar será:

$$\mathcal{L}(\chi_D, \chi_K; \mathcal{H}) = w_d \mathcal{L}_d(\chi_D, \chi_K; \mathcal{H}_d) + w_{cb} \mathcal{L}_{cb}(\chi_D, \chi_K; \mathcal{H}_{cb}) \quad (5.3a)$$

$$\mathcal{L}_d(\chi_D, \chi_K; \mathcal{H}_d) = \frac{1}{|\mathcal{H}_d|} \sum_{x \in \mathcal{H}_d} \left\| f\left(\tau, \xi, \lambda, \frac{\partial \widehat{Se}}{\partial \xi}, \frac{\partial \widehat{Se}}{\partial \tau}, \frac{\partial^2 \widehat{Se}}{\partial \xi^2}, \frac{\partial \widehat{D}}{\partial \xi}, \frac{\partial \widehat{K}}{\partial \xi}\right) - Se_m \right\|_2^2 \quad (5.3b)$$

$$\mathcal{L}_{cb}(\chi_D, \chi_K; \mathcal{H}_{cb}) = \frac{1}{|\mathcal{H}_{cb}|} \sum_{x \in \mathcal{H}_{cb}} \|\mathcal{B}(\hat{u}, x)\|_2^2 \quad (5.3c)$$

Donde D y K son la difusividad y conductividad hidráulica, χ_D, χ_K corresponden al conjunto de pesos y bias para D y K respectivamente, Se es la humedad relativa adimensional.

Para lograr el objetivo de entrenar las redes neuronales informadas por procesos físicos y que estas simulen los valores de la difusividad y la conductividad hidráulicas reemplazando a las curvas de retención se propone el siguiente problema de optimización:

$$\min_{\chi_D, \chi_K} \mathcal{L}(\chi_D, \chi_K; \mathcal{H}) \quad (5.4)$$

8.7. Anexo G, Adimensionalización del modelo de Richards

Para los problemas físicos matemáticos, las unidades de medidas son de suma importancia para que el sistema de ecuaciones tenga coherencia y significado físico, sin embargo, para métodos de resolución numérica, las unidades del sistema pueden aumentar la complejidad y presentar indeterminaciones al momento de buscar soluciones. Es por estas razones que se recomienda adimensionalizar estas ecuaciones, para así aumentar la estabilidad numérica, generalizar términos y facilitar el cálculo. Los siguientes métodos de discretización se realizaron para una ecuación de Richards adimensionalizada. En el presente trabajo se adimensionalizar los componentes espaciales, temporales y la humedad con los siguientes términos.

$$\xi = \frac{z}{H} ; \forall z \in [0, H] \Rightarrow \xi \in [0, 1] \quad (5.17)$$

$$\tau = \frac{t - t^0}{\Delta t} ; \forall t \in [t_0, t_f] \Rightarrow \tau \in [0, 1] \quad (5.18)$$

$$Se = \frac{\theta - \theta_R}{\theta_S - \theta_R} ; \forall \theta \in [0, \theta_S] \Rightarrow Se \in [0, 1] \quad (5.19)$$

Luego se obtiene la ecuación de Richards en términos adimensionalizados a partir de la ecuación (3.8a) con los términos presentados en (5.17), (5.18), (5.19).

$$\frac{H^2 \partial Se}{\Delta t \partial \tau} = \left(\frac{\partial D}{\partial \xi} \right) \left(\frac{\partial Se}{\partial \xi} \right) + D \left(\frac{\partial^2 Se}{\partial \xi^2} \right) - \left(\frac{H}{\Delta \theta} \right) \left(\frac{\partial K}{\partial \xi} \right) \quad (5.20)$$

Aplicando las ecuaciones (5.17), (5.18), (5.19) en las condiciones de borde, y condiciones iniciales (3.8b), (3.8c) y (3.8d) se obtiene lo siguiente:

$$Se(\tau^0) = Se^0 = \frac{(\theta^0 - \theta_r)}{(\theta_s - \theta_r)} ; \forall \xi \in [0, 1] \wedge \tau = 0 \quad (5.21)$$

$$\left(-D \frac{\Delta \theta}{H}\right) \left(\frac{\partial Se}{\partial \xi}\right) + K = R(\tau) ; \forall \xi = 0 \wedge \tau > \tau_0 \quad (5.22)$$

$$\frac{\partial Se}{\partial \xi} = 0 ; \xi = 1 \wedge \tau > \tau_0 \quad (5.23)$$

8.8. Anexo H, Código sensibilización de hiperpárametros

Definir par_solid = [H, T, theta_ini, theta_s, theta_r, Ks]

Definir par_vg = [alpha, n, method]

Definir dim_model = [Dt, Dz]

Definir Z_vect = vector desde 0 hasta H con paso Dz

Definir T_vect = vector desde 0 hasta T con paso Dt

// Generación de datos

(S_m, K_m, D_m) = Sim_CN_VG(par_solid, par_vg, dim_model)

Definir S = S_m

Definir K = K_m

Definir inputs = S[1, :]

Definir max_output = max(K[1, :])

Definir outputs_v = K[1, :]

Definir outputs = K[1, :] / max_output

// Definir función de pérdida

FUNCIÓN modelLoss(net, S, K0)

Definir S_ini = S

```

Definir K_out = forward(net, S_ini)
Definir loss1 = l2loss(K_out, K0)
Retornar loss1 y sus gradientes respecto a net.Learnables
FIN FUNCIÓN

```

```

Definir resultsTable como tabla vacía con columnas [i, j, err]

```

```

// Búsqueda de hiperparámetros

```

```

PARA i DESDE 1 HASTA 10 HACER

```

```

    numBlocks = i

```

```

    PARA j DESDE 1 HASTA 50 HACER

```

```

        fcOutputSize = j

```

```

        Definir fcBlock como [capa fullyConnected(fcOutputSize), capa tanh]

```

```

        Definir layers como [capa input(1), repetir(fcBlock, numBlocks), capa fullyConnected(1)]

```

```

        Definir net_K como dlnetwork(layers)

```

```

        Actualizar net_K con valores double

```

```

        Inicializar solverState = lbfgsState

```

```

        Definir maxIterations = 1500

```

```

        Definir gradientTolerance = 1e-5

```

```

        Definir stepTolerance = 1e-5

```

```

        Definir X = darray(inputs, "CB")

```

```

        Definir X0 = darray(outputs, "CB")

```

```

        Definir accfun = dlaccelerate(modelLoss)

```

```

        Definir lossFcn = función que evalúa accfun con net_K, X, X0

```

```

// Entrenamiento

```

```

iteration = 0

```

```

MIENTRAS iteration < maxIterations HACER

```

```

    iteration = iteration + 1

```

```

    (net_K, solverState) = lbfgsupdate(net_K, lossFcn, solverState)

```

```
SI solverState.GradientsNorm < gradientTolerance O solverState.StepNorm <
stepTolerance O solverState.LineSearchStatus == "failed" ENTONCES
```

```
SALIR BUCLE
```

```
FIN SI
```

```
FIN MIENTRAS
```

```
Mostrar [i, j]
```

```
Definir X_test = dlarray(inputs, "CB")
```

```
Definir X0_test = outputs
```

```
Definir predictions = predict(net_K, X_test)
```

```
Definir K_test_pred = extraer datos de predictions
```

```
Definir err = norma((K_test_pred - X0_test)) / norma(X0_test)
```

```
Agregar fila {i, j, err} a resultsTable
```

```
SI err < 1e-3 ENTONCES SALIR BUCLE
```

```
FIN PARA
```

```
SI err < 1e-3 ENTONCES SALIR BUCLE
```

```
FIN PARA
```

```
Guardar resultsTable en 'Resultados_K.xlsx'
```

```
// Graficar resultados
```

```
predictions = predictions * max_output
```

```
CREAR FIGURA 2
```

```
Graficar(X_test, predictions, "x")
```

```
Graficar(X_test, outputs_v, "o")
```

```
Etiquetar ejes y título
```

```
Definir SSres = sum((X0_test - K_test_pred)^2)
```

```
Definir SStot = sum((X0_test - mean(X0_test))^2)
```

```
Definir R2_K = 1 - (SSres / SStot)
```

CREAR FIGURA 1

Scatter plot de $X0_{test}$ vs K_{test_pred}

Graficar línea de referencia en rojo

Etiquetar ejes y título con R^2

// Repetir proceso para D_m en lugar de K_m

Repetir los mismos pasos anteriores, pero reemplazando K con D

Guardar resultados en 'Resultados_D.xlsx'

Graficar resultados de D de la misma manera

8.9. Anexo I, Código entrenamiento y optimización DNN

Para la red con dos salidas:

INICIALIZAR parámetros físicos del modelo: $H = 550$ cm // Altura de la pila $T = 44$ días // Tiempo de simulación $\theta_{ini} = 0.14$ // Humedad inicial $\theta_s = 0.33$ // Humedad de saturación $\theta_r = 0$ // Humedad residual $d\theta = \theta_s - \theta_r$ $K_s = 170$ cm/día // Conductividad hidráulica de saturación $\alpha = 0.035$ // Parámetro VGM $n = 2.267$ // Parámetro VGM method = 2 // Método VGM $Dt = 1/24$ // Paso de tiempo (1 hora) $Dz = 10$ cm // Paso espacial

CALCULAR vectores de espacio y tiempo: $Z_{vect} = 0:Dz:H$ $T_{vect} = 0:Dt:T$

GENERAR datos de simulación: $[S_m, K_m, D_m] = \text{Sim_CN_VG}(\text{par_solid}, \text{par_vg}, \text{dim_model})$

OBTENER dimensiones del modelo: $[N_z, N_t] = \text{tamaño}(S_m)$

SELECCIONAR datos de entrada: $Se_{in} = S_m(1, :)$ $D_{in} = D_m(1, :)$ $K_{in} = K_m(1, :)$

NORMALIZAR datos: $\max_D = \text{máximo}(D_{in})$ $D_{in} = D_{in} / \max_D$ $\max_K = \text{máximo}(K_{in})$ $K_{in} = K_{in} / \max_K$

SEPARAR datos en conjuntos de entrenamiento y validación: $\text{train_Idx} = \text{seleccionar aleatoriamente } 70\% \text{ de los índices}$ $\text{Val_Idx} = \text{índices restantes}$

ASIGNAR datos a conjuntos: $Se_{train}, D_{train}, K_{train} = \text{subconjuntos de entrenamiento}$ $Se_{val}, D_{val}, K_{val} = \text{subconjuntos de validación}$

TRANSFORMAR datos de validación a vectores columna

DEFINIR arquitectura de la red neuronal: numBlocks = 2 fcOutputSize = 10 Definir bloque fcBlock con capa totalmente conectada y activación tanh Crear capas de la red con capas de entrada, bloques y salida net_k_d = dlnetwork(layers) Convertir parámetros de la red a tipo double

DEFINIR función de pérdida: function [loss, gradients] = modelLoss(net_k_d, S, K0, D0) Forward pass de la red con entrada S Obtener salidas K_out y D_out Calcular pérdidas loss1 y loss2 (L2 loss) Total loss = loss1 + loss2 Calcular gradientes con respecto a los parámetros de la red

CONFIGURAR parámetros de optimización: solverState = lbfgsState maxIterations = 1500 gradientTolerance = 1e-5 stepTolerance = 1e-6

CONVERTIR datos de entrenamiento a dldarray

DEFINIR función de evaluación de pérdida con aceleración

INICIALIZAR monitor de entrenamiento

INICIAR bucle de entrenamiento: Mientras iteración < maxIterations y no detener monitor: Actualizar iteración Actualizar red neuronal con lbfgsupdte Guardar pérdida de entrenamiento Calcular predicción en datos de validación Calcular pérdida de validación Actualizar métricas del monitor Si gradiente < tolerancia o paso < tolerancia o falla búsqueda de línea: Detener entrenamiento

GUARDAR red entrenada

GENERAR gráficos de desempeño del entrenamiento: Figura 1: pérdida de entrenamiento vs validación Figura 2: scatter real vs predicho para K en entrenamiento Figura 3: histograma de errores en K Figura 4: scatter real vs predicho para D en entrenamiento Figura 5: histograma de errores en D

VALIDACIÓN FINAL EN DATOS DE VALIDACIÓN: Repetir evaluación con datos de validación y generar gráficos correspondientes

Ahora para las redes separadas

INICIO

// Definir parámetros físicos y de simulación Definir H, T, theta_ini, theta_s, theta_r, Ks, alpha, n, method, Dt, Dz Crear vectores Z_vect y T_vect basados en H, T, Dz y Dt

// Generación de datos de simulación Llamar a Sim_CN_VG(par_solid, par_vg, dim_model) para obtener S_m, K_m, D_m

// Preparación de datos para entrenamiento y validación Extraer Se_in, D_in, K_in desde S_m, D_m, K_m Normalizar D_in y K_in Dividir datos en conjuntos de entrenamiento y validación

// Inicialización de la red neuronal para K Definir arquitectura de la red con numBlocks y fcOutputSize Crear capa de entrada y capas ocultas fullyConnectedLayer, tanhLayer Crear objeto de red neuronal net_k

```

// Función de pérdida para la red de K Definir modelLoss_K Predecir K_out usando la red Calcular
pérdida loss1 con l2loss Calcular gradientes con dlgradient

// Configuración del optimizador Inicializar solverState, maxIterations, tolerancias y estructuras de
datos Acelerar cálculo de pérdida con dlaccelerate y definir función lossFcn Inicializar monitor de
entrenamiento

// Entrenamiento de la red para K Para cada iteración hasta maxIterations o condición de parada:
Actualizar pesos con lbfgsupdate Calcular y almacenar pérdida de entrenamiento Calcular y
almacenar pérdida de validación Actualizar monitor de entrenamiento Verificar condiciones de
parada Fin Para

Guardar modelo entrenado DNN_K

// Inicialización de la red neuronal para D Repetir los pasos anteriores con una nueva red net_d

// Función de pérdida para la red de D Definir modelLoss_D de forma similar a modelLoss_K

// Entrenamiento de la red para D Repetir el proceso de entrenamiento con net_d

Guardar modelo entrenado DNN_D

// Validación y evaluación de los modelos Calcular predicciones para los conjuntos de
entrenamiento y validación Calcular coeficiente de determinación R2 para K y D Generar gráficos
de dispersión entre valores reales y predichos Generar histogramas de errores para evaluar la
distribución de errores

FIN

```

8.10. Anexo J, Código comparación de modelos DNN

INICIO

LIMPIAR memoria, variables y cerrar gráficos

```

// Definir parámetros del modelo

DEFINIR H = 550 // Altura de la pila en cm

DEFINIR T = 44 // Tiempo de simulación en días

DEFINIR theta_ini = 0.14 // Humedad inicial (cm³/cm³)

DEFINIR theta_s = 0.33 // Humedad de saturación (cm³/cm³)

DEFINIR theta_r = 0 // Humedad residual (cm³/cm³)

```

```

DEFINIR dtheta = theta_s - theta_r
DEFINIR Ks = 170 // Conductividad hidráulica de saturación (cm/día)
DEFINIR alpha = 0.035 // Parámetro VGM (1/cm)
DEFINIR n = 2.267 // Parámetro VGM
DEFINIR method = 2 // Método VGM
DEFINIR Dt = 1/24 // Paso de tiempo (día, cada hora)
DEFINIR Dz = 10 // Paso espacial en cm

// Agrupar parámetros en arreglos
DEFINIR par_solid = [H, T, theta_ini, theta_s, theta_r, Ks]
DEFINIR par_vg = [alpha, n, method]
DEFINIR dim_model = [Dt, Dz]

// Generar vectores de tiempo y espacio
DEFINIR Z_vect = rango(0, Dz, H)
DEFINIR T_vect = rango(0, Dt, T)

// Simulación base usando el modelo numérico
(S_m, K_m, D_m) = Sim_CN_VG(par_solid, par_vg, dim_model)

// Cargar redes neuronales entrenadas
CARGAR "DNN_K_D.mat"
CARGAR "DNN_K.mat"
CARGAR "DNN_D.mat"

net_k_d = DNN_K_D[1]
max_K1 = DNN_K_D[2]
max_D1 = DNN_K_D[3]

```

```

net_k = DNN_K[1]
max_K2 = DNN_K[2]

net_d = DNN_D[1]
max_D2 = DNN_D[2]

// Simulación con redes neuronales
(S_m1, K_m1, D_m1) = Sim_CN_DNN_1(DNN_K_D, par_solid, dim_model)
(S_m2, K_m2, D_m2) = Sim_CN_DNN_2(DNN_K, DNN_D, par_solid, dim_model)

// Graficar resultados de simulaciones
GRAFICAR superficie de S_m con T_vect y Z_vect
GRAFICAR superficie de S_m1 con T_vect y Z_vect
GRAFICAR superficie de S_m2 con T_vect y Z_vect

// Comparación de resultados con redes neuronales

// Evaluar error para red con K y D juntos
SSres = SUMA_CUADRADOS(S_m - S_m1)
SStot = SUMA_CUADRADOS(S_m - PROMEDIO(S_m1))
R2_S_1 = 1 - (SSres / SStot)

CONVERTIR S_m y S_m1 a vectores
GRAFICAR dispersión de S_m vs S_m1 con línea ideal y mostrar R²

// Evaluar error para red con K y D separados
SSres = SUMA_CUADRADOS(S_m - S_m2)

```

$SS_{tot} = \text{SUMA_CUADRADOS}(S_m - \text{PROMEDIO}(S_{m2}))$

$R2_S_2 = 1 - (SS_{res} / SS_{tot})$

CONVERTIR S_m y S_{m2} a vectores

GRAFICAR dispersión de S_m vs S_{m2} con línea ideal y mostrar R^2

FIN

8.11. Anexo K, Código entrenamiento y optimización PINN

// Definir parámetros de simulación, Definir $H = 550$ // Altura de la pila en cm, Definir $T = 44$
//Tiempo de simulación en días, Definir $\theta_{ini} = 0.14$ // Humedad inicial (cm³/cm³), Definir
 $\theta_s = 0.33$ // Humedad de saturación (cm³/cm³), Definir $\theta_r = 0$ // Humedad residual
(cm³/cm³), Definir $d\theta = \theta_s - \theta_r$, Definir $K_s = 170$ // Conductividad hidráulica de
saturación (cm/día), Definir $Dt = 1/24$ // Intervalo de tiempo en días, Definir $Dz = 10$
//Discretización espacial en cm

Definir $D_{chi} = 10$

Definir $D_{tau} = 1/24$

Definir $D_H = 24$ // Conversión día/hora

Definir $A = 308$ // Área de irrigación en m²

// Definir vectores de espacio y tiempo

Definir $Z_{vect} = [0:Dz:H]$

Definir $T_{vect} = [0:Dt:T]$

Definir $N_z = \text{Redondear}(H/Dz)$

Definir $N_t = \text{Redondear}(T/Dt)$

Definir $\text{Param} = [Dt, D_{tau}, H, D_{chi}, d\theta]$

Definir Q_0_{vect} como vector de valores de caudal de ingreso

// Simulación de S_m , K_m , D_m

$[S_m, K_m, D_m] = \text{Sim_CN_VG}(\text{par_solid}, \text{par_vg}, \text{dim_model})$

// Inicialización de la red neuronal

```

numBlocks = 2
fcOutputSize = 20
Definir fcBlock como:
fullyConnectedLayer(fcOutputSize)
tanhLayer
Definir layers como:
featureInputLayer(1)
repmat(fcBlock, [numBlocks, 1])
fullyConnectedLayer(2)
softmaxLayer
net_k_d = dlnetwork(layers)
net_k_d = dlupdate(@double, net_k_d)
// Función de pérdida y cálculo de gradientes
FUNCION modelLoss(net_k_d, Se_m_full, Nz, Nt, Q_0_vect, Param)
Inicializar Se_sim con ceros de tamaño (Nz+1, Nt+1)
Se_sim(:, 1) = Se_m_full(:, 1)
Definir n_d = tamaño de Q_0_vect
Definir D_H = 24
Definir A = 308
PARA d DESDE 1 HASTA n_d
    Definir q0 = Q_0_vect(d,1)
    Definir Rj = (q0 / A) * 100
    PARA h DESDE 1 HASTA D_H
        Definir k = (d - 1) * D_H + h
        SI k == 1 ENTONCES
            CONTINUAR
        FIN SI
    Definir Se_prev = Se_sim(:, k - 1)

```

```

Se_prev = dlarray(Se_prev, 'CB')
Y_1_2 = forward(net_k_d, Se_prev)
Definir D = extraer datos de Y_1_2(1,:)
Definir K = extraer datos de Y_1_2(2,:)
Se_sim(:, k) = euler_Forward(Se_prev, D, K, Rj, Param)
FIN PARA
FIN PARA
Se_sim = dlarray(Se_sim, 'CB')
Definir loss = l2loss(Se_sim, Se_m_full)
Definir Learnables = net_k_d.Learnables
Definir tabla de gradientes con valores calculados
RETORNAR loss, gradients
FIN FUNCION

// Optimización de la red neuronal solverState = lbfgsState maxIterations = 1500 gradientTolerance
= 1e-7 stepTolerance = 1e-7

accfun = dlaccelerate(@modelLoss) lossFcn = @(net_k_d) dlfeval(@modelLoss, net_k_d,
Se_m_full, Nz, Nt, Q_0_vect, Param)

monitor = trainingProgressMonitor(Metrics="TrainingLoss", Info=["Iteration", "GradientsNorm",
"StepNorm", "Loss"], XLabel="Iteration")

iteration = 0 MIENTRAS iteration < maxIterations Y NO monitor.Stop iteration = iteration + 1
[net_k_d, solverState] = lbfgsupdate(net_k_d, lossFcn, solverState)

Actualizar información en monitor

SI solverState.GradientsNorm < gradientTolerance O solverState.StepNorm < stepTolerance O
solverState.LineSearchStatus == "failed" ENTONCES

    Mostrar "Optimización Fallida."

    SALIR

FIN SI
FIN MIENTRAS

Mostrar "Optimización completada." Guardar modelo como "DNN_K_D"

FIN

```

8.12. Anexo L, Código comparación de modelos matemáticos

INICIO

LIMPIAR memoria, variables y cerrar gráficos

// Definir parámetros del modelo

DEFINIR H = 550 // Altura de la pila en cm

DEFINIR T = 44 // Tiempo de simulación en días

DEFINIR theta_ini = 0.14 // Humedad inicial (cm^3/cm^3)

DEFINIR theta_s = 0.33 // Humedad de saturación (cm^3/cm^3)

DEFINIR theta_r = 0 // Humedad residual (cm^3/cm^3)

DEFINIR dtheta = theta_s - theta_r

DEFINIR Ks = 170 // Conductividad hidráulica de saturación (cm/día)

DEFINIR alpha = 0.035 // Parámetro VGM (1/cm)

DEFINIR n = 2.267 // Parámetro VGM

DEFINIR method = 2 // Método VGM

DEFINIR Dt = 1/24 // Paso de tiempo (día, cada hora)

DEFINIR Dz = 10 // Paso espacial en cm

DEFINIR Nchi = 55 // Elementos finitos en chi

DEFINIR Ntau = 24 // Elementos finitos en tau

// Agrupar parámetros

DEFINIR par_solid = [H, T, theta_ini, theta_s, theta_r, Ks]

DEFINIR par_vg = [alpha, n, method]

DEFINIR dim_model = [Dt, Dz]

// Vectores de espacio y tiempo

```

DEFINIR Z_vect = rango(0, Dz, H)
DEFINIR T_vect = rango(0, Dt, T)

// Simulación base usando el modelo numérico
(S_m, K_m, D_m) = Sim_CN_VG(par_solid, par_vg, dim_model)
Se_m_full = S_m
K_m_full = K_m
D_m_full = D_m

// Parametrización adimensionalizada
DEFINIR Dt = 1 // Día
DEFINIR tau = linspace(0,1,Ntau)
DEFINIR chi = linspace(0,1,Nchi)
DEFINIR Dtau = 1/Ntau
DEFINIR Dchi = 1/Nchi
DEFINIR Param = [Dt, Dtau, H, Dchi, dtheta]

DEFINIR Q_0_vect =
[12,19,19,0,0,0,12,32,36,39,46,27,23,24,27,30,29,28,28,27,36,35,26,26,26,25,22,26,24,24,26,22,
22,25,27,27,23,19,20,19,16,17,21,30] // Vector de caudales

// Determinar vectores para tau y chi
DEFINIR chi_vect = rango(0, Dchi, 1)
DEFINIR Matriz_v_tau como lista de tamaño T
DEFINIR tau_vect_1d = rango(0, Dtau, 1)

PARA i = 1 HASTA T HACER
    SI i == T ENTONCES
        Matriz_v_tau[i] = tau_vect_1d

```

```

SI NO
    Matriz_v_tau[i] = tau_vect_1d SIN el último elemento
FIN SI
FIN PARA

DEFINIR tau_vect = concatenar_horizontalmente(Matriz_v_tau)

// Inicializar matriz para guardar Se simulados por día
DEFINIR Matriz como lista de tamaño T
DEFINIR Se_sim como matriz de ceros de tamaño (Nchi+1, Ntau+1)

DEFINIR Se_ini = S_theta(theta_ini, theta_r, theta_s, 0)
DEFINIR Se_ini_v = vector de tamaño (Nchi+1, 1) con valores Se_ini
ASIGNAR Se_sim[:, 1] = Se_ini_v // Condición inicial

DEFINIR A = 308 // Área de irrigación en m²

SI Dtau <= ((H^2)/(2*(Dt*24)))*(Dchi^2) ENTONCES // Criterio de estabilidad CFL
    PARA k = 1 HASTA T HACER
        DEFINIR q0 = Q_0_vect[k] // Caudal de ingreso en el día k
        PARA j = 1 HASTA Ntau HACER // Paso del tiempo en días
            DEFINIR Rj = (q0 / A) * 100 // Tasa de riego en cm/d

            DEFINIR Se_prev = Se_sim[:, j] // Estado previo

            // Calcular parámetros hidrodinámicos
            DEFINIR h = h_theta(Se_prev, n, alpha, Nchi, method)
            DEFINIR K_prev = K_theta(Se_prev, Ks, n, Nchi, method)

```

```

DEFINIR dtheta_dh = dtheta_dh_theta(h, theta_s, theta_r, n, alpha, Nchi, method)
DEFINIR D_prev = D_theta(K_prev, dtheta_dh, Nchi)

// Aplicar método de Euler explícito
ASIGNAR Se_sim[:, j+1] = euler_Forward(Se_prev, D_prev, K_prev, Rj, Param)
FIN PARA

// Guardar resultados y actualizar condiciones iniciales
SI k == T ENTONCES
    Matriz[k] = Se_sim
SI NO
    Matriz[k] = Se_sim[:, 1:Ntau]
    DEFINIR Se_ini_k = Se_sim[:, end]
    Se_sim = matriz de ceros de tamaño (Nchi+1, Ntau+1)
    Se_ini_k = vector de tamaño (Nchi+1, 1) con valores Se_ini_k
    ASIGNAR Se_sim[:, 1] = Se_ini_k // Nueva condición inicial
FIN SI
FIN PARA

DEFINIR Se_full = concatenar_horizontalmente(Matriz) // Unir resultados de todos los días
SI NO
    MOSTRAR 'Condición de estabilidad no cumplida'
FIN SI

// Problema: pueden aparecer valores complejos debido a que Se supera 0.6 en el segundo día

// Graficar resultados
GRAFICAR superficie de Se_full con tau_vect y chi_vect

```

GRAFICAR superficie de Se_m_full con T_vect y Z_vect

FIN

8.13. Anexo M, Código Implementación PINN alternativo

COMIENZO

// --- Parámetros Físicos del Sistema ---

DEFINIR H = 550 // Altura de la columna (cm)

DEFINIR T = 44 // Tiempo total de simulación (días)

DEFINIR theta_ini = 0.14 // Contenido de agua inicial

DEFINIR theta_s = 0.33 // Contenido de agua saturado

DEFINIR theta_r = 0 // Contenido de agua residual

DEFINIR dtheta = theta_s - theta_r

DEFINIR Ks = 170 // Conductividad hidráulica saturada (cm/día)

DEFINIR A = 308 // Área transversal (m²)

// --- Parámetros de Discretización ---

DEFINIR Dz = 10 // Tamaño del paso espacial (cm)

DEFINIR Dt = 1/24 // Tamaño del paso temporal (días, es decir, 1 hora)

DEFINIR D_H = 24 // Factor de conversión: horas por día

// --- Calcular Dimensiones de la Malla ---

DEFINIR Z_vect = [0, Dz, 2Dz, ..., H] // Puntos de la malla espacial

DEFINIR T_vect = [0, Dt, 2Dt, ..., T] // Puntos de la malla temporal

DEFINIR Nz = H / Dz // Número de nodos espaciales

DEFINIR Nt = T / Dt // Número de nodos temporales

REDONDEAR Nz al entero más cercano

// --- Empaquetar parámetros en un vector para pasarlo a funciones ---

DEFINIR Param = [Dt, Dtau, H, Dchi, dtheta] // (Nota: Dtau y Dchi se definen en otro lugar)

// --- Datos de Caudal de Entrada (Variable de Decisión) ---

// Este es el caudal de entrada conocido para cada día de la simulación.

DEFINIR Q_0_vect = [12, 19, 19, 0, ..., 30] // Un vector de 44 caudales diarios ($\text{m}^3/\text{día}$)

// --- Generar Datos de Entrenamiento Objetivo ---

// Ejecutar un modelo de alta fidelidad para obtener los perfiles de humedad "reales".

LLAMAR Sim_CN_VG(par_solid, par_vg, dim_model)

DEVUELVE S_m, K_m, D_m // Estas son matrices de valores objetivo

// Suponemos que Se_m_full (saturación efectiva) se deriva de S_m.

// Definir una red neuronal feedforward con 2 salidas (D y K)

DEFINIR numBlocks = 2

DEFINIR fcOutputSize = 20

// Definir un bloque de capas: Fully Connected -> Activación Tanh

DEFINIR fcBlock COMO CAPAS:

fullyConnectedLayer(fcOutputSize)

tanhLayer()

// Ensamblar la arquitectura completa de la red

DEFINIR arquitectura de net_k_d CON CAPAS:

InputLayer(1) // Entrada: Saturación Efectiva (Se)

fcBlock repetido numBlocks veces // Capas ocultas

fullyConnectedLayer(2) // Capa de salida: 2 neuronas

```
softmaxLayer()           // Activación de salida (Nota: softplus podría ser más adecuado para
salidas positivas D y K)
```

```
// Inicializar los pesos de la red y convertir todos los parámetros a doble precisión
```

```
INICIALIZAR net_k_d
```

```
CONVERTIR todos los parámetros de net_k_d al tipo double
```

```
FUNCIÓN modelLoss(net_k_d, Se_m_full, Nz, Nt, Q_0_vect, Param):
```

```
    // Esta función simula el sistema usando la red neuronal y calcula la pérdida.
```

```
    INICIALIZAR Se_sim COMO una matriz de (Nz+1) x (Nt+1) de ceros
```

```
    ESTABLECER Se_sim(:, 1) = Se_m_full(:, 1) // Aplicar la condición inicial
```

```
    ESTABLECER n_d = longitud(Q_0_vect) // Número de días a simular
```

```
    PARA índice_día d desde 1 hasta n_d:
```

```
        ESTABLECER q0 = Q_0_vect(d)    // Obtener el caudal de entrada diario
```

```
        ESTABLECER Rj = (q0 / A) * 100 // Convertir caudal a tasa de riego (cm/día)
```

```
    PARA índice_hora h desde 1 hasta D_H (24 horas por día):
```

```
        ESTABLECER k = (d-1)*D_H + h // Calcular el índice de hora absoluto
```

```
    SI k == 1 ENTONCES:
```

```
        // Ya se tiene la condición inicial, no hacer nada
```

```
        CONTINUAR
```

```
    SINO:
```

```
        ESTABLECER Se_prev = Se_sim(:, k-1) // Estado del paso de tiempo anterior
```

```
        CONVERTIR Se_prev a objeto dlarray
```

```
        // PREDECIR D y K usando la red neuronal
```

```

EJECUTAR pasada forward de net_k_d con entrada Se_prev
    DEVUELVE salida Y_1_2 = [D; K] // Predicciones de la red

EXTRAER D y K de dlarray a arrays numéricos
TRANSPONER D y K a vectores columna

// PROPAGAR el modelo fisico un paso de tiempo hacia adelante
LLAMAR euler_Forward(Se_prev, D, K, Rj, Param)
    DEVUELVE Se_new // Nuevo estado en el tiempo k
ESTABLECER Se_sim(:, k) = Se_new

FIN SI
FIN PARA
FIN PARA

// CALCULAR PÉRDIDA: Error Cuadrático Medio entre la simulación y el objetivo
CONVERTIR Se_sim a dlarray
ESTABLECER loss = l2loss(Se_sim, Se_m_full) // Suma de diferencias al cuadrado
EXTRAER loss a un valor numérico

// CALCULAR GRADIENTES de la pérdida con respecto a todos los parámetros de la red
OBTENER tabla Learnables de net_k_d (contiene todos los pesos/sesgos)
INICIALIZAR una lista vacía gradients

PARA cada parámetro (matriz de pesos/vector de sesgos) en Learnables:
    EXTRAER el valor numérico del parámetro
    CALCULAR el gradiente de la pérdida con respecto a este parámetro
    ALMACENAR el gradiente en la lista gradients

```

FIN PARA

DEVOLVER loss, gradients

FIN FUNCIÓN

// --- Configurar el Optimizador ---

INICIALIZAR solverState para el algoritmo L-BFGS

DEFINIR maxIterations = 1500

DEFINIR gradientTolerance = 1e-7

DEFINIR stepTolerance = 1e-7

// --- Prepararse para el Entrenamiento ---

ACCELERAR la función modelLoss para computación más rápida

DEFINIR lossFcn = un manejador de función que llama a modelLoss con argumentos fijos (Se_m_full, Nz, Nt, Q_0_vect, Param)

CONFIGURAR un monitor de progreso del entrenamiento para mostrar:

- Iteración
- NormaGradientes (Magnitud de los gradientes)
- NormaPaso (Tamaño de la actualización de parámetros)
- Pérdida (Loss)

// --- Bucle de Entrenamiento ---

ESTABLECER iteration = 0

MIENTRAS iteration < maxIterations Y el entrenamiento no se detenga manualmente:

INCREMENTAR iteration por 1

```

// Paso de Optimización L-BFGS
LLAMAR lbfgsupdate(net_k_d, lossFcn, solverState)
    DEVUELVE red_actualizada, estado_solver_actualizado
ESTABLECER net_k_d = red_actualizada
ESTABLECER solverState = estado_solver_actualizado

// Actualizar el monitor de progreso con los nuevos valores
ACTUALIZAR monitor con:
    Iteración, NormaGradientes, NormaPaso, Pérdida

// Verificar Condiciones de Parada
SI NormaGradientes < gradientTolerance ENTONCES
    IMPRIMIR "Convergido: La norma del gradiente es pequeña."
    ROMPER
FIN SI
SI NormaPaso < stepTolerance ENTONCES
    IMPRIMIR "Convergido: El tamaño del paso es pequeño."
    ROMPER
FIN SI
SI la Búsqueda de Línea falló ENTONCES
    IMPRIMIR "Optimización Fallida."
    ROMPER
FIN SI

FIN MIENTRAS

// --- Acciones Finales ---
IMPRIMIR "Optimización completada."

```

GUARDAR los parámetros de la red neuronal entrenada en un archivo "DNN_K_D.mat"

FIN PROGRAMA